

AENEAS 7 — Design Specification

Compiled specification

Generated 2026-09-14

Contents

1. Design — README.md
2. Overview — overview.md
3. Changes from AENEAS 6 — changes-from-v6.md
4. Roles, Skills, and the Worker Pool — roles.md
5. Governance — governance.md
6. Ledger — Overview — ledger/overview.md
7. Ledger — Constitution — ledger/constitution.md
8. Ledger — Feature Spec (spec.md) —
ledger/feature-spec.md
9. Ledger — Plan (the plan.md bundle) — ledger/plan.md
10. Ledger — Tasks (tasks.md) — ledger/tasks.md
11. Ledger — Data Model — ledger/data-model.md
12. Ledger — Clarification Session — ledger/clarification.md
13. Ledger — Amendment — ledger/amendment.md
14. Pipeline — Overview — pipeline/overview.md
15. Pipeline — Phase 1: Specify — pipeline/phase-1-specify.md
16. Pipeline — Phase 2: Plan — pipeline/phase-2-plan.md

17. Pipeline — Phase 3: Schedule —
pipeline/phase-3-schedule.md
18. Pipeline — Phase 4: Build — pipeline/phase-4-build.md
19. Pipeline — Phase 5: Integrate & Release —
pipeline/phase-5-integrate-release.md
20. Pipeline — Phase 6: Operate —
pipeline/phase-6-operate.md
21. Platform — Overview — platform/overview.md
22. Platform — Turn Queue and Workers —
platform/turn-queue.md
23. Platform — Event Store — platform/event-store.md
24. Platform — Projections — platform/projections.md
25. Platform — Concurrency — platform/concurrency.md
26. Platform — Verification — platform/verification.md
27. Platform — Database Schema —
platform/database-schema.md
28. Platform — Recovery — platform/recovery.md
29. Workflows — Main Workflow —
workflows/main-workflow.md
30. Workflows — Clarification — workflows/clarification.md
31. Workflows — Review and Rework —
workflows/review-and-rework.md
32. Workflows — Blocked — workflows/blocked.md
33. Workflows — Abandonment —
workflows/abandonment.md
34. Workflows — Amendment — workflows/amendment.md
35. Workflows — Data Model Change —
workflows/data-model-change.md

- 36. Skills — skills/README.md
- 37. SKILL — skills/plan/SKILL.md
- 38. SKILL — skills/review-data/SKILL.md

Design

README.md

The canonical AENEAS 7 specification. Read [overview.md](#) first.

Directory / file	Covers
overview.md	The system in one document: loops, workers, streams, gates
changes-from-v6.md	Decisions record: each v6 concept and its v7 disposition
roles.md	Roles as skill bundles plus permission sets; the worker pool; human roles
governance.md	Platform-enforced rules, loop bounds, system invariants, principles
ledger/	Artifact formats: Constitution, Feature Spec, Plan bundle, Tasks, Data Model, Clarification, Amendment
pipeline/	The six phases, each as a declared loop with a gate
platform/	Turn queue, event streams, projections, concurrency, verification, schema, recovery
workflows/	Happy path and alternate flows
skills/	The skill catalog and the exemplar plan skill

Overview

overview.md

AENEAS 7 is a governed delivery platform built as a set of declared loops. This document is the whole system in one place; every section links to the document that owns the detail.

The shape

Humans (UI)
| intent, approvals, answers
te

Workers (pool, stateless)
| claim turn → load skill → act → comple

▼ ▼

PLATFORM

Custodian — validates every signal, writes content + event + turns
in one transaction per stream

Streams — one event stream per Feature / Constitution / Data Model

Turn queue — the work owed to roles; claimed with leases

Verifiers — sandboxed tests, contracts, scans, analyze gate, risk score

Scheduler — priority, predecessors, blockers, WIP, write-set conflicts

Merge queue — one integration at a time onto the trunk

Projectors — async, checkpointed read models for queues and dashboards

PostgreSQL: entity tables (aggregate state) · events · turns · content

The platform is the sole writer. Workers and humans signal; the platform validates, writes, and records. No one else evaluates a gate or writes a status.

Artifacts

Artifact	File form	Answers	Authored by role	Approved by
Charter	charter.md	What is this project, who is it for, what must it never do	analyst (interview with the Product Owner)	Product Owner
Constitution	constitution.md	The non-negotiable principles and the vocabulary every Plan must conform to	architect	Solution Architect, Enterprise Architect
Feature Spec	spec.md	What, for whom, why; how we will know it is done	analyst	Product Owner
Plan	plan.md + research.md, data-model.md, contracts/, quickstart.md, threat-model.md, checklists/	How, within the Constitution	planner	reviewers (architecture, data, security, UX), then Solution Architect
Tasks	tasks.md	The ordered, parallelizable units of build work	planner	with the Plan

Artifact	File form	Answers	Authored by role	Approved by
Data Model	one document per entity / enumeration / overview	The project's logical data design as approved and built	platform transcribes from approved Plans	with the Plan; independent changes have their own review
Clarification	a session log inside the artifact it clarifies	The questions asked and the answers that resolved them	analyst / planner / reviewer	the human or role that answered
Amendment	amendments/A-####.md	A governed change to the Constitution	architect	Solution Architect, Enterprise Architect

Every artifact is versioned append-only in the content store and rendered into the feature's working tree under `specs/F-####-slug/` so that skills and coding tools read it as files. See [ledger/](#).

Loops

A loop is a generator turn, a verifier, a bound, and a stop. The six phases are six loops, and the whole feature is one outer loop bounded by its budget.

Phase	Generator (role · skill)	Verifier	Bound	Stop / escalation
1 Specify	analyst · specify, clarify	Spec format check; no unresolved [NEEDS CLARIFICATION]; Product Owner approval	MAX_CLARIFY_SESSIONS, MAX_SPEC_REJECTIONS	Product Owner decides: revise intent or abandon
2 Plan	planner · plan, tasks	Analyze gate; Constitution check; data-model gates; parallel review round; Solution Architect approval	MAX_PLAN_REJECTIONS, MAX_REVIEW_QUESTIONS per reviewer per version	Solution Architect decides: redirect, amend the Constitution, or abandon
3 Schedule	platform scheduler	Gate conditions (approved, predecessors done, unblocked, WIP, write set)	none (deterministic)	Delivery Lead may reorder or force
4 Build	tester · test; implementer · implement	Sandboxed verifier suite per task; frozen acceptance tests	MAX_VERIFY_ATTEMPTS per task, MAX_BUG_RECURRENCE per feature	Recurring failure becomes a Defect to the planner; then a Solution Architect review

Phase	Generator (role · skill)	Verifier	Bound	Stop / escalation
5 Integrate & Release	integrator · integrate, release	Rebase onto trunk, full verifier suite, risk band, deploy health	MAX_INTEGRATION_ATTEMPTS	Conflict or red trunk raises a blocker to the Delivery Lead; HIGH band waits for the Solution Architect
6 Operate	operator · observe, triage	Anomaly rules	none	An anomaly becomes a new Feature; no self-remediation

See [pipeline/](#). Bounds are counted by the platform and refused at the signal; see [governance.md](#).

Turns and workers

A **turn** is the unit of work the platform owes a role. It names the role, the skill, the feature, the triggering event, the artifact scope, a budget, and a deadline. Turns are enqueued by the Custodian in the same transaction as the event that calls for them. A **worker** claims a turn it can run, is issued a token good only for that turn, loads the skill, does the work through the /v1 API, and completes or fails the turn. There are no per-role processes and no notifications to agents.

Five properties of a turn are what make work survive load, and each one is a failure v6 could not prevent because it had no unit of work to attach them to:

Property	Consequence
At most one live turn per (stream, kind, artifact)	Two workers never author one artifact
Exactly one committing write per turn	A turn cannot leave an artifact half-created
Checkpoints stored as it goes	A queued turn resumes instead of re-deriving
A heartbeat that answers CURRENT / SUPERSEDED / CANCELLED	A stale turn stops before its next model call
A lease sized from the skill , heartbeated on a reserved connection	A lapsed lease means a dead worker, not a busy platform

See [platform/turn-queue.md](#) and [roles.md](#).

Streams and projections

Every event belongs to a stream — a Feature, the Constitution, the Data Model, or the platform — and streams are written with optimistic concurrency, so parallel features never wait on each other. Gates read aggregate state on the entity tables, in the same transaction. Projections are asynchronous read models rebuilt from a checkpoint. See [platform/event-store.md](#) and [platform/projections.md](#).

Where parallel features meet

There are exactly three shared resources across features, and each has one structural rule:

Shared resource	Rule
The Data Model lexicon	A name is reserved by the first Plan to declare it at submit; a second is refused. BREAKING changes are refused at the gate and go through the Data Model's own review
The Constitution	Amended rarely; an applied Amendment enqueues a re-analyze turn for every Plan not yet integrated
The codebase	Every Plan declares its write set; the scheduler holds overlapping features; every feature builds on its own branch; the integrator merges one feature at a time onto the trunk

See [platform/concurrency.md](#).

Verification and risk

A worker's completion signal starts verification; it never ends it. The platform runs the verifier suite for the task — build, lint, type-check, unit tests, frozen acceptance tests, contract tests, scanners, checklist assertions — in a sandbox against the feature branch. Failure returns the task; success completes it. At integration the platform computes a risk score from deterministic signals and maps it to a band; MEDIUM and HIGH wait for a Solution Architect. See [platform/verification.md](#).

Governance in one paragraph

Rules are structural or they are not rules. The platform: is the only writer; issues every identifier; owns every status; counts every bound; runs every verifier; enforces every role permission on a per-turn token; keeps the event store and content versions append-only; and refuses, with a reason, anything that does not fit. The list is in [governance.md](#).

Changes from AENEAS 6

changes-from-v6.md

This is the decisions record for the seventh design. Each section takes one of the concerns raised against v6, states the v7 decision, and records why. The final section is a concept-by-concept disposition table so that a reader who knows v6 can find where every v6 mechanism went.

The concerns, as raised:

1. The Technical Analyst commissioning other agents took many serialized turns.
2. Twelve dedicated agents may not make sense against a pool of workers with skills; the roster failed under load.
3. Story and Spec were free-form prose; Spec Kit's fixed formats would be better and align with loop engineering.
4. The single event table and synchronous projections limited concurrency and throughput once several features ran in parallel.
5. What else should change.

Concern 2 has two halves that are worth separating.

Throughput — how much work the system can carry — is §2 and §4. Reliability — agents failing to create specs, claim tasks, and update artifacts once several features ran at once — is §5, which analyses each of those three operations and the mechanisms that replace them.

1. Commissions are replaced by one authoring turn and one parallel review round

v6. The Technical Analyst authored the Spec, but every specialist input — the data model, the threat model, an architecture note, an interaction model, a deployment note — was a **Commission**: a request record the specialist started, authored against, delivered, and the Technical Analyst then read, incorporated into a new Spec version, and settled. Each Commission was at least three agent turns plus two notifications, and the Spec could not enter review until all were settled. A feature that needed data, security, and architecture input spent roughly a dozen agent turns and several hours of wall-clock latency before a reviewer read anything. Every one of those turns was also a fresh model context reconstructing the same Spec.

v7. The **planner** role produces the entire **Plan bundle** in one turn: `plan.md`, `research.md`, `data-model.md`, `contracts/`, `quickstart.md`, `threat-model.md`, and `tasks.md`, each authored by the corresponding skill (the same skills a specialist would have used) inside that turn, with sub-agents where the skill wants them. Specialist expertise enters as a **review**, not as authorship: when the bundle is submitted, the platform first runs the **analyze gate** (a deterministic cross-artifact check plus a model-run consistency pass) and, if that passes, opens one **parallel review round** — architecture, data, security, and UX when the feature has UI — each reviewer returning a structured checklist verdict in a single turn. A verdict may carry a **proposed patch** (a diff against the bundle) so that the planner's revision turn applies rather than re-derives. The

human Solution Architect is asked only once the round is clean.

	v6 (data + security + architecture input)	v7
Authoring turns before review	Spec draft; 3 × (start, author, deliver); 3 × incorporate; submit ≈ 13	Plan bundle: 1
Serialized hops	≈ 8	2 (author → review round)
Review turns	3–4, each able to pause the Spec with questions	3–4 in parallel, one verdict each, questions bounded per round
Machine gates before a human sees it	Commission settlement, data-model gates	Analyze gate (coverage, constitution, ambiguity, consistency) plus the data-model gates

These are design counts, not measurements.

What is kept from the Commission model. The property that mattered was never who typed the data model; it was that the design the platform acts on is **machine-readable where it is approved**. That survives intact: `data-model.md` carries the same structured declarations, validated against the Constitution's representation contract at submit, classified by the platform (a BREAKING change is still refused at the gate), reserved by name across features, and transcribed into the project Data Model **by the platform** at integration — see §6. The specialist that would have authored it now reviews it with the same skill, in an independent context, and can reject or patch it. Author-reviewer separation is enforced structurally: a review turn is always a fresh turn with no access to the authoring turn's context (see [platform/turn-queue.md](#)).

What is dropped. The Commission tables, its eight statuses, its two counters, the settlement gate, incorporation records, withdrawal cascades, the Commission read carve-out, and the `DATA_MODEL_UPDATE` task. Data Model Provenance is restated as "every declaration was reviewed by the data reviewer role before approval", which the sign-off record already proves.

2. A worker pool executing role skills replaces the twelve agents

v6. Each role was one long-lived process behind a webhook: the platform POSTed a notification, the agent's handler ran the turn, and the agent kept its own in-flight guards, retry backoff, reconciliation sweeps, and a latched model client. Twelve of these ran in one host. Under load, a burst of features serialized behind whichever role was on the critical path, the sweeps re-entered flows that were already running, and adding capacity meant adding another copy of a stateful process that would then race the first one.

v7. The platform owns a **turn queue**. Whenever an event calls for work by a role, the Custodian enqueues a **turn** in the same transaction as the event. A **worker** is a stateless runtime that claims a turn it is able to run (FOR UPDATE SKIP LOCKED, lease, heartbeat), receives a **per-turn token** scoped to that turn's role, feature, and artifact, loads the named **skill**, executes, and completes or fails the turn. Any worker can run any role for which it has the skill loaded and the model configured. Capacity is the number of workers; nothing else changes when it grows.

A **role** is therefore two things: a permission set the Custodian enforces on every write made with the turn's token, and a skill bundle the worker loads. It is not a process, not a name, and not a conversation. The v6 principle "Guild roles are teams, not individuals" becomes literal.

Does separation of roles still matter? Yes, for one reason only: separation of duties. The party that authors must not be the party that verifies, and the party that implements must not

be the party that writes the acceptance tests. Those are properties of turns — a review turn is issued fresh, with its own context, under a role the authoring turn's token could not have used — and stateless workers give them for free. Everything else that made roles distinct in v6 (a codename, a webhook, a sweep, a latched client) was runtime accident.

Which v6 roles survive as skills. The analyst (specify, clarify), the planner (plan, tasks), four reviewers (architecture, data, security, UX), the implementer (backend and frontend skill variants), the tester, the integrator (integrate, release), the operator (observe, triage). The Scrum Master's scheduling and blocker routing become deterministic platform logic with a human override (§6). The System Architect's AGGREGATE_DELTA is gone with changeDelta (§6). The Business Analyst's charter interview is the analyst's charter skill.

Load behavior. The queue is the backpressure surface: depth per role is the load metric, WIP limits per phase bound what enters it, and a turn that cannot be claimed within its deadline is a visible, alertable condition rather than a silent stall inside an agent process. Model outages latch a **skill configuration**, not a worker, so one bad model id stands down one skill's turns while every other role keeps running. See [platform/turn-queue.md](#).

3. Artifacts take Spec Kit's formats

v6. The Story and the Spec were governed as to ownership, status, and citations, but their content was prose against a section list. Reviewers argued about proportionality, the platform could check capability identifiers and entity citations and nothing else, and "is this Spec complete" was a judgment call every time.

v7. The three delivery artifacts and the project constitution adopt [Spec Kit](#)'s formats, with AENEAS's governance layered on top rather than replacing them:

v6 artifact	v7 artifact	Format	Owner role	Approver
Archetype	Constitution	constitution.md: numbered Articles carrying MUST/SHOULD principles, plus the AENEAS sections the platform reads (capability map, data design language, security model)	architect	Solution Architect + Enterprise Architect
Story	Feature Spec	spec.md: prioritized user stories (P1/P2/P3) with independent-test notes and Given/When/Then scenarios, FR-###, SC-###, Key Entities, Assumptions, [NEEDS CLARIFICATION] markers, a ## Clarifications session log	analyst	Product Owner
Spec	Plan (bundle)	plan.md (technical context, constitution check, project structure, write set, complexity tracking) with research.md, data-model.md, contracts/, quickstart.md, threat-model.md, checklists/	planner	reviewers, then Solution Architect

v6 artifact	v7 artifact	Format	Owner role	Approver
Tasks	Tasks	tasks.md: T###, [P] parallel marker, [US#] story label, phases (Setup, Foundational, per-story, Polish), exact file paths, checkpoints	planner	approved with the Plan

Why this is more than a template choice:

- **The format is a verifier.** Because the sections are fixed and the identifiers are numbered, the platform's analyze gate can compute coverage (every FR-### mapped to at least one T###, every user story to at least one acceptance scenario and one task), detect unresolved [NEEDS CLARIFICATION] markers, and flag constitution MUST violations as CRITICAL before any reviewer or human spends a turn. Prose cannot be checked; a form can.
- **Clarification is a bounded session, not a status.** The clarify skill scans the nine-category ambiguity taxonomy, asks at most five questions per session as multiple choice or short answer, and writes the answers back into the spec's ## Clarifications section and the sections they resolve. A human answers a form, not a thread. The [CLARIFY] and [UPDATED] statuses disappear; a feature at [DRAFT] with an open session simply reports waiting_on.
- **Independent slices.** Spec Kit's rule that each user story is an independently testable increment gives AENEAS a unit smaller than the feature to verify, ship, and roll back, and a natural parallelism unit for implementer turns ([P] tasks in different files).
- **Coding agents already understand it.** The bundle is rendered onto the feature branch under specs/F-0042-slug/, so any Spec Kit-aware tool, and any implementer skill, reads it as files in the working tree. The content store stays the source of truth; the working tree is a projection of it.
- **Loop engineering alignment.** In a loop the verifier is the bottleneck, not the generator. Fixed formats with numbered,

testable statements are what make verifiers writable: SC-### become checks, Given/When/Then scenarios become acceptance tests, contracts/ become contract tests, checklists/ become review rubrics.

The full formats are in [ledger/](#).

4. Event streams, asynchronous projections, and write-set scheduling

v6. One events table with all references inside a JSONB payload, one per-artifact advisory lock per write, and every projection updated inside the write transaction (Custodian step 7). Correct, and adequate for one feature at a time. Under parallel features the costs compounded: every write paid for every projection it touched; the ready board rewrote every row of a feature on each task event; expression indexes over payload keys were the only way to find anything; and the outbox, the sweeps, and the reaper all polled the same tables. Beyond the database, parallel features collided where no lock existed at all: in the codebase.

v7. Four changes, each removing one class of contention.

Per-aggregate streams with optimistic concurrency.

Every event belongs to a stream: a Feature, the Constitution, the Data Model, or the platform stream. `events(stream_id, stream_seq)` is unique; a writer reads the stream head, validates, and inserts at `head + 1`. Two writers to the same feature conflict on the unique index and one retries; writers to different features never touch the same rows and never wait. There are no advisory locks. Typed columns (`stream_type`, `stream_id`, `feature_id`, `turn_id`) replace payload expression

indexes. The table is range-partitioned on the global sequence so that growth is an archival concern rather than an index-depth one. See [platform/event-store.md](#).

Gates read aggregate state; projections are read models only. Entity tables carry `current_status` and the columns a gate needs, written in the stream's own transaction — so the gate that decides "may this plan be scheduled" reads a consistent, current row. Projections (queues, boards, dashboards) are built **asynchronously** by projectors that read the global sequence forward from a durable checkpoint. They are never in the write path and never consulted for a governing decision. Rebuilding is resetting a checkpoint. A caller that needs read-your-writes passes the sequence its write returned, and the read waits for the projector to pass it. See [platform/projections.md](#).

A turn queue instead of a notification fan-out. Enqueuing a turn is one row in the same transaction as the event. There is no outbox, no per-agent webhook, no delivery retry ladder, and no roster-wide sweep inferring lost notifications from board state. What replaces the sweep is the queue's own lease: a claimed turn whose lease lapses is requeued. Humans are still notified (the UI), through a projection.

Write-set-aware scheduling and a merge queue. The one shared resource v6 never governed is the code. Every Plan declares its **write set** (paths and modules it will change) in `plan.md`. The scheduler holds a feature whose write set overlaps one already in Build (configurable as a hard hold or as an ordering hint). Each feature builds on its own branch; the **integrator** rebases onto the trunk, runs the whole verifier suite, and merges, one feature at a time. The merge queue is

the single serialization point in the system, and it is the one place serialization is cheap. Data Model names are reserved across features exactly as in v6; the Constitution is amended rarely and an Amendment triggers a re-analyze turn on every open Plan. See [platform/concurrency.md](#).

5. Turns are the unit of work, so authoring, claiming, and updating stop failing under load

Concern 2 named a symptom — the roster failing under load — that is worth separating from throughput. Three specific operations failed in v6 at scale: creating a spec, claiming a task, and updating an artifact. They share one cause, and it is not slowness.

The platform could not see a unit of work. It saw signals arriving and notifications going out, so it could not tell a turn in flight from one that never started, could not stop the same work being started twice, could not resume a half-finished one, and could not bound what one was allowed to spend. Every repair in v6 was therefore an agent-side guard for something only the platform could have seen: an in-memory in-flight set, a per-flow backoff, a roster-wide sweep, a one-shot gate on the charter interview, a forced-reclaim endpoint.

What each operation actually did

Operation	v6 mechanism	Why it failed
Creating a spec	One turn issuing specs/create, then a draft version, then one tasks/create per task, then one commission request per specialist, then submit-for-review — a dozen mutations with no transaction around them	Any failure mid-sequence left partial state: a spec record with no content version, half a task graph, or a Commission the settlement gate would then hold the spec behind. Two entry paths (the notification and the sweep) could start the same turn, and the guard against that was an in-process set, so it did not hold across replicas
Claiming a task	Every board-changing event notified every Guild agent; each read /board and claimed the top row; the Custodian's per-artifact lock let one win	A thundering herd on every event, where the losers returned without trying the next task and then idled until the next event or the 15-minute sweep. Claims on one feature serialized behind a single advisory lock, and the claim heartbeat competed with board reads for the same connection pool — so under load a lease could lapse because the platform was busy, and the reaper would reclaim work that was still being done

Operation	v6 mechanism	Why it failed
Updating an artifact	Status preconditions and the capability allow-list were checked at the write, under a per-artifact lock, after the model call that produced the content	A turn whose reads had gone stale spent its whole model call and was then refused. A unique-constraint collision surfaced as a 500, which the agent client treated as transient and retried. Output above the token ceiling truncated, failed the JSON parse, and saved nothing. The agent host ran every handler as an uncapped background task, so load multiplied concurrency inside one process

What v7 does

Six mechanisms, all of them consequences of the platform owning a **turn**:

Mechanism	Replaces	Effect
One live turn per (stream, kind, artifact), a partial unique index	In-memory in-flight sets, the sweep's re-entrancy guard, STORY-219	A duplicate enqueue is a no-op. Two turns cannot author one artifact, across any number of workers
One committing write per turn , with whole-artifact submission	A dozen sequential mutations	Partial state is unrepresentable. There is no half-created spec to resume, so no resume path to get wrong
A turn per ready task, claimed with SKIP LOCKED	Board fan-out and the claim race	Exactly one worker is handed each task; nobody contends, nobody idles after losing, and no board read precedes a claim
Checkpoints	Discarding a whole turn's output on any failure	A queued attempt resumes from the last stage. A crash costs minutes, not a full planner turn's tokens
Heartbeat currency (CURRENT / SUPERSEDED / CANCELLED, with remaining budget)	Discovering staleness at the final write	A superseded turn stops before its next model call. Wasted spend is bounded by one heartbeat interval instead of one turn

Mechanism	Replaces	Effect
In-turn schema validation with a bounded repair pass	Truncation → parse failure → fail-closed turn → identical retry	A malformed output costs one small call. Only a non-converging repair fails the turn

Two supporting changes close the conditions that made lease expiry lie. Leases are sized from each skill's declared `expected_seconds` rather than one global constant, and the heartbeat path is a single indexed-row update served from a reserved connection allowance, so it can never queue behind a projection read. A lapsed lease means a dead worker, which is what the reaper assumes.

Backpressure is added where v6 had none:

`MAX_OPEN_FEATURES` caps features in flight at intake, `slots` cap concurrent turns per worker, and both surface as queue depth rather than as contention. v6 bounded only Build.

Finally, the condition becomes visible. The `pen_holder` projection reports, for every artifact with a live turn, the turn, its skill version, the worker, its heartbeat age, its checkpoints, and its spend. An agent working, an agent crashed, and an agent that never received its notification were indistinguishable in v6 — all three looked like an artifact that had stopped moving.

6. Further changes

Platform-run verifiers replace `changeDelta`. v6 had each Guild Lead score its own work on a Fibonacci rubric, then a System Architect task RSS-combined four scores into a band. Self-assessed risk is not governance, and it cost a scanning state machine, a re-claim, and an extra task per feature. v7 computes a **risk score** from deterministic signals the platform already holds — diff size, modules touched, contract surface changed, data-model `change_kind`, new dependencies, scanner findings, acceptance test delta — and maps it to a band that decides whether a human reviews the integration. Tests, contracts, build, lint, and scans run as **verifiers** in a sandbox on every implementation-complete signal; a failing verifier returns the task, and a worker cannot override it. See [platform/verification.md](#).

Scheduling is platform logic. The Scrum Master's ordering rules were deterministic (priority, predecessors, blockers, WIP) and are now computed by the platform, with write-set conflicts added. A human **Delivery Lead** role may reorder or force. Blockers are triaged by rule (machine-resolvable → cleared; otherwise → the human role that owns the blocked artifact's phase), so no model turn sits between a blocker and the person who can clear it.

The analyze gate. Before any review turn is enqueued, the platform runs Spec Kit's analyze pass over the bundle: duplication, ambiguity, underspecification, constitution alignment, coverage gaps, inconsistency. CRITICAL findings return the bundle to the planner without spending a reviewer or a human. This is where most of v6's review-round rejections are caught for free.

The Data Model is transcribed by the platform. A Plan's `data-model.md` declarations are approved with the Plan and written into the project Data Model by the platform when the feature integrates. No task, no turn, no agent. The v6 reasons for writing in Build (no residue from unbuilt features, the write is a comparison against the approved declaration) hold better when the platform does the comparison itself.

Blocked is a condition, not a status. v6 threaded [BLOCKED] through every artifact's status table. v7 keeps blockers in a registry and derives a blocked flag; an artifact's status says where it is in its lifecycle, and the registry says whether it can move. Fewer transitions, no resume edge cases.

Petition and Amendment collapse into Amendment. One artifact, one lifecycle: proposed → under review → approved → applied. The proposal and the change record were two views of one decision.

Budgets are a governed bound. Each feature carries a token and cost budget; every turn reports usage; exhaustion raises a blocker to the Product Owner. This is the loop-engineering "stop" for the whole feature loop. Telemetry beyond the bound stays non-governing.

Per-skill model routing. A skill declares a model tier (reasoning, standard, fast); deployment maps tiers to models. Review checklists and triage run on fast models; planning and implementation on reasoning models.

Fewer statuses, fewer humans-in-the-loop, same authority. Humans approve the Feature Spec (Product Owner), the Plan (Solution Architect), the Constitution and its Amendments (Solution Architect and Enterprise Architect), and

high-risk integrations (Solution Architect). Everything else is a machine gate.

Disposition table

v6 concept	v7 disposition
Charter, charter interview	Kept; the analyst's charter skill seeds <code>constitution.md</code>
Archetype (12 subtypes, <code>archetype_meta</code>)	Constitution: one document with numbered Articles and the platform-read sections; versioned whole
Story	Feature Spec (<code>spec.md</code>)
Spec	Plan bundle
Task types <code>DATA_MODEL_UPDATE</code> , <code>IMPLEMENTATION</code> , <code>TEST_CREATION</code> , <code>TEST_EXECUTION</code> , <code>AGGREGATE_DELTA</code>	<code>tasks.md</code> entries of kind <code>setup</code> , <code>foundation</code> , <code>test</code> , <code>implement</code> , <code>polish</code> ; the platform runs execution and verification; no aggregate task
BUG, DEFECT tasks	Kept, raised by the platform on verifier failure after a tester triage turn decides target (implementer or planner)
Commission (all types)	Removed; planner skills author, reviewer skills review in parallel
Clarification (dialogue artifact, @-mentions, <code>[CLARIFY]</code>)	Clarification session: bounded structured questions written back into the artifact; no status
Petition + Amendment	Amendment
Data Model (ledger artifact, declarations, classification, reservation, citation)	Kept; written by the platform at integration; declarations authored in <code>data-model.md</code>

v6 concept	v7 disposition
Twelve named agents, agent host, webhooks, sweeps	Worker pool, turn queue , per-turn tokens, role skills
In-memory in-flight guards, per-flow retry backoff, the one-turn-per-Story fix (STORY-219)	One live turn per (stream, kind, artifact), a partial unique index
Multi-call artifact creation (create → draft → tasks → commissions → submit)	One committing write per turn, whole-artifact submission
Board fan-out and the claim race; projection_active_claims	A turn per ready task, claimed with SKIP LOCKED; turn leases
A single claim_lease_seconds and a heartbeat competing for the pool (STORY-214, STORY-220)	Leases sized from the skill's expected_seconds; heartbeats on a reserved connection allowance
Truncation → parse failure → fail-closed turn	In-turn schema validation with a bounded repair pass
Discovering supersession at the final write	Heartbeat currency: stop before the next model call
Forced reclaim of stuck work (STORY-192/193), "is it working or dead?"	pen_holder projection; a turn past deadline is an alert
Scrum Master	Platform scheduler + human Delivery Lead override
Guild Lead / sub-agent model	A skill may spawn sub-agents inside its turn; the platform sees the turn
changeDelta, RSS, bands, AGGREGATE_DELTA	Platform risk score from deterministic signals; band decides human review at integration

v6 concept	v7 disposition
Scanning states (SCANNING_*, VERIFICATION_FAILED)	Verifier run with two task outcomes: DONE or VERIFICATION_FAILED
Deliverable verification Tier 1 / Tier 2	Verifier suite: platform checks plus checklists/ and quickstart.md assertions
Story Firewall	Removed: implementers read the Feature Spec's scenarios because the tests derive from them. Replaced by Test Independence : acceptance tests are authored from spec and contracts before implementation and frozen; an implementer diff touching them fails verification
Spec Fidelity, QA Independence, Approved Before Build	Kept in turn-token scope: a turn may read and write only its feature's artifacts, in the statuses its role permits
Archetype Supremacy (capability identifiers)	Kept as Constitution Supremacy , enforced by the analyze gate and the capability check at submit
Sign-off workflow, staged agent-then-human	Kept; agent stage is one parallel review round; invalidation on new version kept
MAX_REJECTION_COUNT, MAX_REVIEW_QUESTION_ROUNDS, MAX_COMMISSION_*, MAX_BUG_RECURRENCE_COUNT	Restated as declared loop bounds in governance.md; commission bounds dropped

v6 concept	v7 disposition
Backlog, ready board, sign-off tracker, blockers, scanning failures projections	Backlog and board become queue projections for humans; the turn queue is a table; blockers registry kept; scanning failures folded into verifier-run failures
Notification outbox, drain	Removed; turn queue rows are the obligation
Active claims, stale-claim reaper, heartbeat	Turn leases and heartbeats; the reaper requeues turns
Idempotency keys	Kept
WIP limit at the scheduling gate only (STORY-216)	Kept, plus MAX_OPEN_FEATURES at intake and slots per worker
Execution provenance, turn telemetry, model price, efficiency report	Kept, simplified: every turn records usage; budgets are governed; the rest is non-governing
Abandonment	Kept; cascades along the feature stream
Operations anomaly, escalation routing, security escalation	Operator skill records anomalies; routing is by rule to the owning human; a fix is a new feature
Greek codenames	Dropped; skills are named by what they do

Roles, Skills, and the Worker Pool

roles.md

A **role** is a permission set plus a skill bundle. A **worker** is a stateless runtime that executes one **turn** at a time as whatever role the turn names. Nothing in the system is a long-lived agent.

Why roles are not processes

v6 ran one process per role. That made the role's identity, its webhook, its retry state, its model client, and its reconciliation sweep all the same object, so the platform could only scale a role by duplicating a stateful process that then raced its twin. It also meant a role on the critical path was a single lane.

v7 keeps what a role is for — a bounded set of permissions and a bounded set of competences — and discards the process. The Custodian enforces the permission set on every write made with a turn token, and the worker enforces the competence by loading the skill the turn names. Any worker with the skill installed and the model tier configured can take any turn of that role. Adding a worker adds capacity to every role at once.

Separation of duties survives at the turn level: the platform never issues a review turn to the context that authored what is under review, because every turn is a fresh context. See platform/turn-queue.md.

Role catalog

Role	Skills	Reads	Writes (through the API, within the turn's feature scope)	Model tier
analyst	charter, specify, clarify	Intent, Charter, Constitution, Feature Spec	Charter content; Feature Spec versions; clarification questions to the Product Owner; submit-for-review	reasoning
planner	plan, tasks, defect	Feature Spec (approved), Constitution, Data Model, Plan bundle, review verdicts, verifier reports	Plan bundle versions (all files), tasks.md, clarification questions to the analyst, submit-for-review, Defect resolutions	reasoning
architect	constitution, amend	Charter (approved), Constitution, Amendments, Plans (for impact)	Constitution versions, Amendment proposals and content	reasoning
reviewer.architecture	review-architecture	Plan bundle, Constitution, Data Model	Review verdict with checklist and optional patch; bounded questions	standard

Role	Skills	Reads	Writes (through the API, within the turn's feature scope)	Model tier
reviewer.data	review-data	Plan bundle, Constitution §Data Design Language, Data Model	Review verdict with checklist and optional patch; bounded questions	standard
reviewer.security	review-security	Plan bundle, Constitution §Security Model, threat-model.md	Review verdict with checklist and optional patch; bounded questions	standard
reviewer.ux	review-ux	Plan bundle, Feature Spec scenarios, Constitution §Interface Conventions	Review verdict with checklist and optional patch; bounded questions	standard
tester	test, triage	Feature Spec, contracts/, quickstart.md, tasks.md (test tasks); verifier reports	Acceptance and contract test files on the feature branch; Bug/Defect triage decisions	standard

Role	Skills	Reads	Writes (through the API, within the turn's feature scope)	Model tier
implementer	implement, implement-frontend, implement-data, fix	Plan bundle, tasks.md, Feature Spec scenarios, the feature branch, verifier reports	Code, migrations, unit tests on the feature branch; task claim, implementation-complete; blockers	reasoning
integrator	integrate, release	The feature branch, the trunk, verifier reports, risk report	Rebase/merge resolution on the feature branch; deployment signals	standard
operator	observe, triage-anomaly	Telemetry, deployment records	Anomaly records	fast
analyzer	analyze	Feature Spec, Plan bundle, Constitution	Analyze report	fast

Model tiers are deployment-mapped names, not model ids. A skill may raise its tier for a turn kind (a fix on a third attempt may escalate to reasoning); it may never lower another role's.

The analyzer role exists so that the analyze gate is a turn like any other — queued, leased, budgeted, and auditable — rather than logic hidden in the Custodian. Its deterministic passes run in the platform; its consistency pass is the turn.

What a role may never do

These hold regardless of skill, because the Custodian enforces them on the turn token:

Rule	Enforcement
Write outside the turn's feature	The token carries <code>feature_id</code> ; a write naming another feature's artifact is refused
Write an artifact its role does not own	The permission set names the artifact types and statuses the role may write
Write status	No endpoint accepts a status; every transition is the Custodian's response to a validated signal
Act after the turn ends	The token expires at turn completion, failure, or lease lapse
Author and review the same version	A review turn is refused for a version whose authoring turn ran under a token from the same worker session; workers are stateless so this only guards a misconfigured deployment

Rule	Enforcement
Modify frozen acceptance tests	The verifier fails an implementer diff that touches paths frozen at the tester's task completion
Exceed the turn budget	The worker reports usage on heartbeat; the platform fails the turn at the budget and records why
Make a second committing write	The Custodian marks the turn's write when it commits and refuses the next one, naming what landed
Keep working after being superseded	The heartbeat answers SUPERSEDED or CANCELLED; a completion from such a turn is refused

Human roles

Humans act only through the UI, which calls the Custodian on their behalf. Every human decision is an event.

Human role	Approves	Decides	Reads
Product Owner	Charter, Feature Spec	Answers clarification sessions; abandons a Feature; releases a budget hold	Everything on their features except the working tree
Solution Architect	Plan, Constitution, Amendment	Reviews MEDIUM/HIGH risk integrations; resolves a recurring-failure escalation	Everything
Enterprise Architect	Constitution, Amendment	—	Constitution, Amendments, Charter
Delivery Lead	—	Reorders or forces scheduling; clears judgment blockers; retries a failed turn; abandons a task	Queues, board, blockers, turns, budgets
Admin	—	Operates the platform: worker registry, skill versions, model tiers, projection rebuilds	Operational surfaces

There is no human role that authors an artifact. A Product Owner who wants a different Feature Spec answers a clarification or rejects with a reason; the analyst revises.

Skills

A skill is a versioned directory:

skills/<name>/	
SKILL.md	front matter (name, role, version, model_tier, inputs, outputs, tools, verifier, budget) + the instructions
templates/	artifact templates the skill fills (Spec Kit templates where one exists)
checklists/	the rubric a reviewer skill answers, item by item

The platform records the skill name and version on every turn, so a change to a skill is visible in the audit trail of every artifact produced after it. Skills are repository content, versioned with the design, and the worker refuses to run a turn naming a skill version it does not have.

The catalog and an exemplar are in [skills/](#).

The worker

A worker is a process that:

1. Advertises the roles it can run (the skills it has loaded and the model tiers it can reach).
2. Claims one turn at a time from the queue for those roles.
3. Receives the turn's token, budget, and artifact references.
4. Materializes the feature's working tree (branch checkout plus the rendered specs/`F-####-slug/` bundle) when the skill needs a filesystem.
5. Executes the skill, calling the `/v1` API for every read, validating each model output against the schema the skill declares and running a bounded repair pass on a malformed one, and checkpointing at each declared stage.
6. Heartbeats with usage, and stops immediately if the heartbeat says the turn is superseded or cancelled.
7. Makes the turn's single committing write, then completes — or fails with a class the platform can route on.

A worker holds no state between turns. Two workers are interchangeable; a worker that dies loses at most the work since its last checkpoint, and the turn requeues at lease expiry. See [platform/turn-queue.md](#).

Inside a turn, a skill may run sub-agents, parallel tool calls, or a Spec Kit command. The platform sees the turn's reads, writes, usage, and result. It never sees the sub-agents, which is the v6 Guild Lead abstraction kept at the only level it was ever needed.

Governance

governance.md

Rules the platform enforces. A rule a skill could ignore is not on this page.

Rules

Rule	Enforcement
Platform-Owned Transitions	No API accepts a status. Every transition is executed by the Custodian in response to a validated signal, recorded as an event on the artifact's stream, and mirrored on the entity table
Sole Writer	Workers and the UI hold no database credentials. All writes pass through the Custodian
Turn Scope	Every worker write carries a turn token naming turn_id, role, feature_id, and expiry. A write outside that scope, or after expiry, is refused
Single Committing Write	A turn performs any number of reads and exactly one committing write. A second is refused, naming the write that landed. Checkpoints and heartbeats are exempt because they change no governed state. A turn therefore cannot leave an artifact half-created

Rule	Enforcement
Whole-Artifact Submission	An artifact is submitted entire — the Plan bundle with its tasks.md, the spec with its sections — never assembled by successive calls. This is what makes Single Committing Write affordable
One Live Turn	At most one QUEUED or CLAIMED turn exists per (stream, kind, artifact), enforced by a partial unique index. A duplicate enqueue is a no-op, so no artifact is ever authored twice concurrently
Currency	A turn learns at its heartbeat that it is superseded or cancelled, and its completion is refused with that reason. Work does not continue against an artifact that has moved
Append-Only Audit	events and every content version table reject UPDATE and DELETE by trigger
ID Issuance	Identifiers (F-####, T### within a feature, A-####) are issued by the platform from sequences. A worker that supplies one is refused

Rule	Enforcement
Format Conformance	A submitted spec.md, plan.md, tasks.md, data-model.md, or constitution.md is parsed against its template. A missing required section, an unnumbered requirement, a task without a file path, or an unresolved [NEEDS CLARIFICATION] at submit-for-review is refused with the location
Constitution Supremacy	A Plan that relies on a capability, technology, or pattern the Constitution does not declare, or that violates a MUST Article, is refused at submit by the analyze gate's deterministic pass and again by its consistency pass as CRITICAL. A gap is closed by an Amendment, never by a line in the Plan
Coverage	At submit, every FR-### maps to at least one T###, every user story to at least one acceptance scenario and one task, every SC-### to a verifier assertion or a checklist item. Unmapped items refuse the submission
Data Model Citation	Every entity a Plan cites exists in the Data Model at [APPROVED]; a citation of a deprecated or retired document is refused with the reason

Rule	Enforcement
Data Model Reservation	A (kind, name) a Plan introduces is reserved from its submit-for-review; a second Plan introducing the same pair is refused, naming the holder. The reservation lapses on abandonment or on a version that drops it, and is consumed at integration
Data Model Classification	change_kind (ADDITIVE, STRUCTURAL, BREAKING) is computed by the platform from the declaration diff, at submit and again at integration. BREAKING is refused at both; it travels the Data Model's independent review with an impact list
Test Independence	Acceptance and contract tests are authored by a tester turn from the Feature Spec and contracts/ and frozen when that task completes. An implementer diff touching a frozen path fails verification
Verified Completion	No task completes on a worker's signal. Implementation-complete starts the task's verifier suite in a sandbox; only a passing run completes the task. A failing run is not human-overridable
Approved Before Build	A task is claimable only when its feature's Plan is at [SCHEDULED] or [IN_BUILD] and every predecessor task is [DONE]

Rule	Enforcement
Write-Set Discipline	A Plan declares its write set. An implementer diff outside it fails verification unless the task's entry in <code>tasks.md</code> names the path; the scheduler holds a feature whose write set overlaps a feature in Build when the policy is strict
Serialized Integration	One feature integrates onto the trunk at a time. Integration requires a green full verifier run on the rebased branch and, for MEDIUM/HIGH risk, a recorded Solution Architect decision
Governed Constitution Change	Constitution content changes only by applying an [APPROVED] Amendment, exactly once; the platform transitions the Amendment to [APPLIED] in the same transaction and enqueues a re-analyze turn for every Plan not yet integrated
Governed Abandonment	Only a human retires work: the Product Owner a Feature, the Delivery Lead or Product Owner a task. A reason is required; the cascade is strictly downward; [ABANDONED] has no edge out
Bounded Loops	Every loop bound below is a platform counter checked at the signal; reaching it refuses the next iteration and enqueues the declared escalation

Rule	Enforcement
Budgeted Features	Every turn reports usage; the platform fails a turn at its turn budget and raises a BUDGET_EXHAUSTED blocker at the feature budget. Nothing else in the decision path reads a cost
Bounded Intake	Features not yet terminal are capped at MAX_OPEN_FEATURES; intent past the cap is queued as PENDING_INTAKE with its position. Backpressure exists before Build, not only at the scheduling gate
No Self-Remediation	The operator role writes anomaly records only. A production change is a new Feature

Loop bounds

Bound	Counted on	Default	On reaching it
MAX_CLARIFY_SESSIONS	Feature Spec, per version lineage	3	The analyst submits with stated assumptions; the Product Owner decides on them
MAX_CLARIFY_QUESTIONS	Per session	5	Session closes
MAX_SPEC_REJECTIONS	Feature Spec	3	Feature held; Product Owner chooses revise-intent or abandon
MAX_REVIEW_QUESTIONS	Per reviewer per Plan version	2	Reviewer decides on what is present
MAX_PLAN_REJECTIONS	Plan	3	Plan held; Solution Architect chooses redirect, Amendment, or abandon
MAX_ANALYZE_RETURNS	Plan version lineage	3	Plan held for the Solution Architect with the analyze report; the planner is not re-run
MAX_VERIFY_ATTEMPTS	Per task	3	Task held; a Defect is raised to the planner carrying the verifier reports

Bound	Counted on	Default	On reaching it
MAX_BUG_RECURRENCE	Per feature	3	The platform raises a Defect instead of a Bug and adds a Solution Architect review to the integration gate
MAX_INTEGRATION_ATTEMPTS	Per feature	2	Blocker to the Delivery Lead with the conflict report
MAX_TURN_ATTEMPTS	Per turn	3	Turn [FAILED]; the failure is visible in the queue and retryable by the Delivery Lead
TURN_BUDGET	Per turn, from the skill	skill-defined	Turn [FAILED] with BUDGET
FEATURE_BUDGET	Per feature, from intent capture	project default	BUDGET_EXHAUSTED blocker to the Product Owner, who raises the budget or abandons
MAX_BUILD_WIP	Features in Build	project default	Scheduler holds
MAX_OPEN_FEATURES	Features not [DONE] or [ABANDONED]	project default	New intent is queued as PENDING_INTAKE

Bound	Counted on	Default	On reaching it
MAX_REPAIR_PASSES	Per model call within a turn	2	The turn fails TRANSIENT with the invalid output attached; the next attempt resumes from the last checkpoint
MAX_CHECKPOINTS, MAX_CHECKPOINT_BYTES	Per turn, from the skill	skill-defined	The checkpoint write is refused; the turn continues

System invariants

The platform halts a feature's progression and surfaces the condition if any of these is false.

1. Every integrated change traces to a [DONE] task, to an [APPROVED] Plan version, to an [APPROVED] Feature Spec version, to a recorded intent.
2. Every Plan approved conforms to the Constitution version current at its approval, and every Plan not yet integrated has been re-analyzed against every Amendment applied since.
3. Every stream is gap-free: `stream_seq` is contiguous per stream.
4. No task is [CLAIMED] or [VERIFYING] with a predecessor not [DONE].
5. Every [DONE] task has a passing verifier run recorded against the version it completed.
6. No acceptance-test path frozen for a feature differs between the tester's completing version and the integrated version.
7. Every Data Model version traces to the approved Plan version whose declaration it transcribes, or to its own recorded review.
8. No two Plans past submit-for-review introduce the same (kind, name).
9. Every turn token ever honored named a turn that was [CLAIMED] at the time.
10. No turn has more than one committing write recorded against it.

11. No (stream, kind, artifact) has two live turns against it.
 12. Every applied Amendment corresponds to exactly one Constitution version.
 13. No feature exceeds FEATURE_BUDGET without a BUDGET_EXHAUSTED blocker recorded.
-

Principles

The platform is the loop harness. Generators are skills; verifiers, bounds, stops, and escalations belong to the platform. A skill can be wrong; the loop around it cannot be skipped.

Governance is structural. If a rule can be bypassed by ignoring an instruction, it is not governed. Instructions live in skills and shape quality; rules live in the Custodian and shape what is possible.

Formats are verifiers. A fixed, numbered artifact format is the cheapest verifier there is: it makes coverage, ambiguity, and conformance computable.

Roles are permissions, workers are capacity. Separation of duties is a property of turns, not of processes.

Streams isolate; the merge queue serializes. Features never wait on each other in the database. They wait on each other only where they truly share something — a name, a principle, a file — and each of those has one explicit rule.

Self-assessment is telemetry. What a worker says about its own work is recorded and never decides anything. What the verifier says decides.

A turn is the unit the platform can see. Because work is a row the platform issues, leases, and closes, it can be deduplicated, resumed, budgeted, cancelled, and shown to a human. Every property this design relies on for reliability under load — one live turn per artifact, one committing write, resume from checkpoint, stop when superseded, a visible pen holder — follows from the platform knowing what a unit of work is. v6's repairs were agent-side guards for a unit the platform could not see.

Humans approve intent and accept risk. Product Owners approve what is to be built; Solution Architects approve how and accept high-risk integrations; Enterprise Architects approve the constitution. Everything else is a machine gate, and humans see it only when a bound is reached.

Ledger — Overview

ledger/overview.md

The Ledger is the set of governed artifacts. Content lives in the platform's content store as append-only versions; the platform renders each feature's artifacts into its working tree so skills and tools read them as files.

Artifacts

Artifact	Scope	File form	Owner role	Stream
Charter	project (singleton)	charter.md	analyst	constitution
Constitution	project (singleton)	constitution.md	architect	constitution
Amendment	project, many	amendments/A-### #.md	architect	constitution
Data Model	project, one document per entity, enumeration, or the overview	data-model/<kind>/<name>.md	platform (transcribed) or reviewer.data (independent change)	data_model
Feature Spec	feature (one)	specs/F-####-slug/spec.md	analyst	feature
Plan bundle	feature (one)	specs/F-####-slug/plan.md, research.md, data-model.md, contracts/ quickstart.md, threat-model.md, checklists/	planner	feature
Tasks	feature (one)	specs/F-####-slug/tasks.md	planner	feature

Artifact	Scope	File form	Owner role	Stream
Clarification session	inside the artifact it clarifies	## Clarifications section	the asking role	feature (or constitution)
Review verdict	per reviewer per Plan version	reviews/<role>-v <n>.md	reviewer roles	feature
Verifier report	per task per attempt	platform-generated	platform	feature

Versioning

Every submission is a new version row. The current version is the newest. Approval pins a version: `approved_version_id` on the entity is what every downstream reader of authorized content uses, never "the latest".

A bundle (the Plan) versions as a whole: one version row carries every file's content, so a reviewer approves a consistent set. Individual files are not versioned separately.

Rendering into the working tree

For each feature the platform maintains a branch `feat/F-####-slug`. At the start of every turn that needs a filesystem, the worker materializes the branch and the platform writes the current (or, in Build, the approved) artifact versions under `specs/F-####-slug/`. Those files are read-only to the skill; a skill produces a new artifact version by submitting through the API, not by committing the file. The commit that carries the rendered bundle onto the branch is the platform's, so the branch history records which artifact version each code commit was built against.

This is what makes Spec Kit-shaped skills, and any Spec Kit-aware tool, work unchanged: they find `spec.md`, `plan.md`, and `tasks.md` where they expect them.

Reference keys

Artifact	Key	Issued
Feature	F-####	at intent capture, from feat_seq
Feature Spec, Plan, Tasks	borrow the feature's key: F-0042 spec / plan / tasks	—
Task	T###, unique within the feature (F-0042/T017)	by the platform when tasks.md is accepted
Amendment	A-####	from amnd_seq
Data Model document	(kind, name)	the name is the identifier
Turn	UUID	at enqueue

UUIDs are the identity everywhere in storage and the API; keys are derived on read and resolved on input.

Status vocabulary

Fewer statuses than v6, because blocked is a condition and clarification is a session.

Status	Applies to	Meaning
[DRAFT]	Feature Spec, Plan, Constitution, Amendment	Being authored or revised by the owner role
[ANALYZING]	Plan	Submitted; the analyze gate is running
[UNDER_REVIEW]	Feature Spec, Plan, Constitution, Amendment	Reviewers (agent stage, then human stage) hold open decisions
[APPROVED]	Feature Spec, Plan, Constitution, Amendment	All required decisions recorded as approved; version pinned
[REJECTED]	Feature Spec, Plan, Constitution, Amendment	The stage closed with a rejection; returned to the owner role with every reason
[SCHEDULED]	Plan	The scheduler released the feature into Build; tasks may be claimed
[IN_BUILD]	Plan	The first task was claimed
[INTEGRATING]	Plan	All tasks [DONE]; the feature is in the merge queue
[DONE]	Feature Spec, Plan, Feature	Integrated and released
[APPLIED]	Amendment	Its Constitution change is in force

Status	Applies to	Meaning
[ABANDONED]	Feature and everything under it; Amendment	Retired by a human decision. Terminal

Derived conditions carried on every read, never stored as status: `blocked` (an open blocker names the artifact), `waiting_on` (the role or human whose turn or decision is next), `held_by` (the scheduler's reason for holding a [APPROVED] Plan).

Task statuses are in [tasks.md](#).

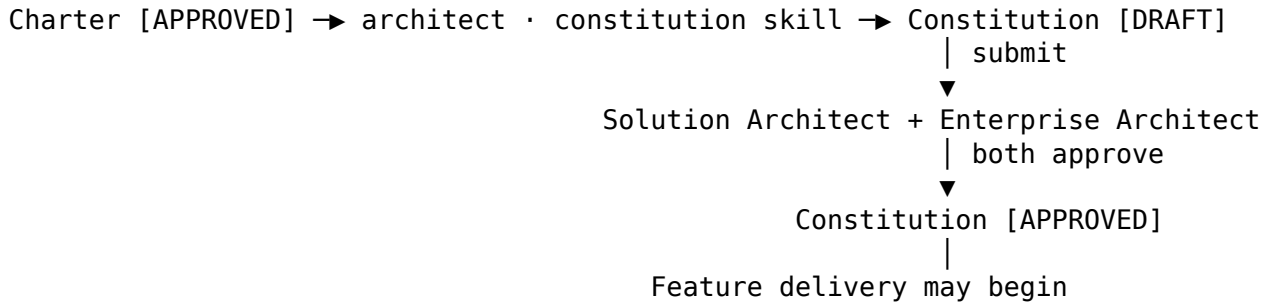
Ledger — Constitution

ledger/constitution.md

The Constitution is the project's architectural constitution: the principles every Plan must satisfy and the vocabulary every Plan may use. It replaces the v6 Archetype and takes Spec Kit's constitution.md shape, extended with the sections the platform reads mechanically.

Owner: architect role **Approvers:** Solution Architect, Enterprise Architect

Role in the pipeline



No Feature may be created until the Constitution is [APPROVED]. After that it changes only by an applied [Amendment](#).

Format

```

# <Project> Constitution

**Version**: 1.3.0 · **Ratified**: 2026-09-01 · **Last amended**: 2026-09-12 (A-0007)

## Core Principles

### Article I – <name>
<principle stated as MUST / MUST NOT / SHOULD, with its rationale>

### Article II – <name>
...

## Capability Map
| Identifier | Capability | Scope | Status |
|---|---|---|---|
| CAP-AUTH | Authentication and session management | ... | GRANTED |

## Technology Stack
| Layer | Choice | Version constraint | Notes |

## Data Design Language
- Naming, identity, representation, constraint and sensitivity conventions
- The **representation contract**: the schema a `data-model.md` declaration must satisfy

## Security Model
- Trust boundaries, control catalog (`CTL-###`), threat surfaces (`TS-###`),
  the `controlCoverage` shape a `threat-model.md` must carry

## Interface Conventions

```

- API style, error shape, UI/accessibility level

Non-Functional Baselines

- | Concern | Baseline | Measured by |

Operational Model

- Environments, deployment, observability, rollback

Integration Landscape

- | System | Contract | Owner |

Compliance Mandates

- | Regulation | Obligations | Evidence |

Approved Patterns and Anti-Patterns

Governance

- How this document changes (Amendment), versioning policy (semver), what a Plan's Constitution Check must answer

Articles carry the MUST language the analyze gate treats as CRITICAL when violated. The remaining sections are the platform-read vocabulary: the Capability Map is what plan.md's ## Required Capabilities is checked against; the Data Design Language's representation contract is what data-model.md declarations are validated against; the Security Model's control catalog is what threat-model.md's coverage block is computed against.

Versioning

The Constitution is versioned as one document. Semantic versioning is stated in the document and enforced by the platform on an Amendment: a change that removes or redefines an Article or a granted capability is MAJOR; one that adds an Article, capability, or constraint is MINOR; wording is PATCH. A MAJOR Amendment enqueues a re-analyze turn on every Plan not yet integrated; a MINOR one does the same only for Plans whose ## Required Capabilities or Constitution Check cites what changed; a PATCH enqueues nothing.

Statuses

Status	Meaning
[DRAFT]	The architect is deriving it from the approved Charter, or revising after rejection
[UNDER_REVIEW]	Submitted; both human decisions open
[APPROVED]	In force. Further change is by Amendment only

Status	Meaning
[REJECTED]	A human rejected; returned with every reason

Readership

Every role and every human reads the Constitution. Authoring is the architect's alone; approval is the two humans'.

Ledger — Feature Spec (spec.md)

ledger/feature-spec.md

The Feature Spec is the entry point of all work: structured human intent, the "what" and "why", written so that its success is measurable and its stories independently testable. It replaces the v6 Story and takes Spec Kit's spec.md format.

Owner: analyst role **Approver:** Product Owner

Format

Feature Specification: <name>

****Feature****: F-0042 · ****Branch****: feat/F-0042-<slug> · ****Status****: [DRAFT]
****Priority****: 2 · ****Type****: BUSINESS | TECHNICAL · ****Created****: 2026-09-12
****Input****: <the Product Owner's intent, verbatim>

User Scenarios & Testing

User Story 1 – <title> (Priority: P1)
<narrative: as a <role>, I want <capability>, so that <benefit>>
****Why this priority****: ...
****Independent test****: <how this story alone is verified and delivers value>
****Acceptance scenarios****:
1. ****Given**** ..., ****When**** ..., ****Then**** ...
2. ...

User Story 2 – <title> (Priority: P2)
...

Edge Cases
- What happens when ...?
- How does the system handle ...?

Requirements

Functional Requirements
- ****FR-001****: The system **MUST** ...
- ****FR-002****: Users **MUST** be able to ...

- ****FR-003****: The system MUST ... [NEEDS CLARIFICATION: <what is unknown>]

Key Entities

- ****<Entity>****: what it represents, key attributes, relationships (no schema)

Success Criteria

- ****SC-001****: <measurable, technology-agnostic outcome>
- ****SC-002****: ...

Constraints

- Measurable constraints stated at intake: limits, retention, accessibility level, what must never be shown. These are acceptance criteria, not solutions.

Assumptions

- Scope boundaries and defaults taken where the input was silent.

Clarifications

Session 2026-09-12

- Q: <question> → A: <answer>

Dependencies

- Predecessors: F-0031

Rules the platform enforces on the format

Check	When	Refusal
Every required section present; at least one user story with priority and at least one acceptance scenario	every submission	names the missing section
FR-### and SC-### numbered contiguously from 001	every submission	names the gap
No [NEEDS CLARIFICATION] marker remains	submit-for-review	lists the markers
No implementation detail in a BUSINESS spec (a technology, a library, a schema)	submit-for-review, analyze pass	HIGH finding returned to the analyst
Every user story has an Independent test	submit-for-review	names the story
Priority 1-5 present	creation	refused

The distinction the v6 Story documented in prose is now a check: a constraint ("responses up to 100 MB must work") is an acceptance criterion and belongs here; a mechanism ("stream to disk above 10 MB") is HIGH here and belongs in the Plan.

Statuses

Status	Trigger	Meaning
[DRAFT]	FeatureCreated; SpecRejected → analyst revision	The analyst is authoring, possibly with an open clarification session
[UNDER_REVIEW]	analyst submit-for-review, format checks pass	The Product Owner holds the decision
[APPROVED]	Product Owner approves	Version pinned; a plan turn is enqueued
[REJECTED]	Product Owner rejects with reasons	Returned to [DRAFT] on the analyst's next version; rejection_count incremented
[DONE]	Plan integrated and released	The intent is realized
[ABANDONED]	Product Owner abandons the Feature	Terminal

Reaching MAX_SPEC_REJECTIONS holds the Feature and asks the Product Owner to revise the intent or abandon.

Clarification

Before submitting, the analyst runs the `clarify` skill: it scans the spec against the nine-category ambiguity taxonomy (functional scope, domain model, interaction flows, non-functional requirements, integrations, edge cases, constraints, terminology, completion signals), and asks the Product Owner at most MAX_CLARIFY_QUESTIONS per session, each multiple-choice or short-answer, each with a recommended answer. Answers are written into `## Clarifications` and applied to the section they resolve. A session is bounded and there are at most MAX_CLARIFY_SESSIONS; past that the analyst states assumptions in `## Assumptions` and submits. See [clarification.md](#).

Readership

The Feature Spec is readable by every role on the feature. The v6 Story Firewall is gone: the acceptance scenarios are the tests, so the tester and implementer read them. What replaces the firewall is Test Independence — the tester authors and freezes acceptance tests before any implementer turn, and an implementer diff that touches them fails verification.

Ledger — Plan (the plan.md bundle)

ledger/plan.md

The Plan is the execution design for one feature: how the approved Feature Spec is realized within the Constitution. It replaces the v6 Spec and takes Spec Kit's plan bundle shape. One planner turn authors the whole bundle; specialist roles review it in one parallel round.

Owner: planner role **Reviewers:** reviewer.architecture, reviewer.data, reviewer.security, reviewer.ux (when the feature has UI) **Approver:** Solution Architect

The bundle

File	Content	Read by the platform for
plan.md	Summary; Technical Context; Constitution Check; Required Capabilities; Project Structure; Write Set; Complexity Tracking; Risks	capability check, write-set scheduling, analyze gate
research.md	Decisions made to resolve NEEDS CLARIFICATION in the technical context: decision, rationale, alternatives	analyze gate (no unresolved items)
data-model.md	Entities cited and entities introduced or revised, as prose and machine-readable declarations satisfying the representation contract	citation, reservation, classification, transcription
contracts/	API contracts (OpenAPI, GraphQL SDL, event schemas) for every interface the feature exposes or changes	contract tests, surface-change signal in the risk score
quickstart.md	The validation scenarios a reviewer or verifier walks to confirm the feature works, as executable steps	verifier assertions

File	Content	Read by the platform for
threat-model.md	Threat surfaces engaged (or determined not engaged), threats, controls, and the controlCoverage block against the Constitution's control catalog	security verifier, risk score
checklists/	requirements.md (spec quality), plus per-reviewer rubrics the review round answers	review verdict structure
tasks.md	The task list — see tasks.md	task records, coverage

The bundle is versioned as a whole. A revision resubmits every file.

`plan.md` **format**

Implementation Plan: <feature>

****Feature****: F-0042 · ****Spec****: spec.md@v3 · ****Constitution****: 1.3.0 · ****Date****: 2026-09-12

Summary

<primary requirement and the technical approach, three to five sentences>

Technical Context

****Language/Version****: ... · ****Dependencies****: ... · ****Storage****: ... · ****Testing****: ...

****Target platform****: ... · ****Project type****: ... · ****Performance goals****: ...

****Constraints****: ... · ****Scale/Scope****: ...

Constitution Check

Article	Status	Note
---------	--------	------

---	---	---
-----	-----	-----

I – ...	PASS	...
---------	------	-----

III – Test-first	PASS	acceptance tests in T004–T009 precede implementation
------------------	------	--

VII – Simplicity	JUSTIFIED	see Complexity Tracking
------------------	-----------	-------------------------

Required Capabilities

- CAP-AUTH

- CAP-NOTIFY

Project Structure

Documentation

Source code

<the concrete tree this feature adds to or changes>

```
## Write Set
- `src/billing/**`
- `src/api/routes/subscriptions.py`
- `migrations/2026_09_*.sql`
- `web/app/(account)/subscriptions/**`

## Complexity Tracking
| Violation | Why needed | Simpler alternative rejected because |

## Risks
- ...
```

Constitution Check must list every Article with PASS, JUSTIFIED (with a Complexity Tracking row), or FAIL; a FAIL refuses submission. **Required Capabilities** lists identifiers copied from the Constitution's Capability Map; an unknown identifier refuses the version. **Write Set** is a list of path globs; the scheduler and the verifier read it.

Proportionality

A section the feature does not engage is settled by a one-line negative determination, never omitted and never elaborated. `threat-model.md` is always present: it names every threat surface in the Constitution with a determination. A reviewer never rejects for brevity; it rejects for a wrong determination or a missing thing a build turn would need.

Statuses

Status	Trigger	Meaning
[DRAFT]	SpecApproved enqueues a plan turn; a rejection or an analyze return	The planner is authoring or revising
[ANALYZING]	planner submit-for-review; format, capability, and data-model gates pass	The analyze gate is running (deterministic passes, then an analyzer turn)
[UNDER_REVIEW]	analyze gate passes	Agent stage: reviewers hold open decisions; then human stage: the Solution Architect
[APPROVED]	all decisions approved	Version pinned; task records created; Data Model reservations confirmed; the feature enters the backlog
[REJECTED]	any reviewer rejects (stage closes when all have decided) or analyze returns CRITICAL	Returned to the planner with every finding and patch; rejection_count incremented (analyze returns count separately)
[SCHEDULED]	the scheduler releases it	Tasks with no predecessors are [READY]
[IN_BUILD]	first task claimed	Build in progress
[INTEGRATING]	all tasks [DONE]	In the merge queue
[DONE]	integrated and released	—

Status	Trigger	Meaning
[ABANDONED]	cascade from the Feature	Terminal

The review round

When the analyze gate passes, the platform enqueues one review turn per required reviewer, in parallel. Each returns a **verdict**:

```

# Review: reviewer.data · plan@v2

**Decision**: APPROVE | REJECT
## Checklist
- [x] R1 Every cited entity resolves against the Data Model
- [ ] R4 Every introduced attribute carries a sensitivity classification – Subscription.
  card_last4 is unclassified
## Findings
| ID | Severity | Location | Summary | Recommendation |
|---|---|---|---|---|
| D1 | HIGH | data-model.md#Subscription | card_last4 unclassified | classify as PII, re
stricted |
## Proposed patch

```

--- data-model.md +++ data-model.md @@ ... @@

- - card_last4: string
- - card_last4: string, sensitivity: PII

A checklist item left unanswered refuses the verdict. A REJECT must carry at least one HIGH or CRITICAL finding. A reviewer that needs an answer asks through a bounded clarification session targeting the planner (MAX_REVIEW_QUESTIONS), then decides. The stage closes when every reviewer has decided; the planner revises against the whole round and the patches at once. Sign-off invalidation on a new version reopens the agent stage.

Relationship to other artifacts

- **Feature Spec:** the Plan maps every FR-### and every user story; the coverage check refuses a submission that leaves one unmapped.
- **Constitution:** the Constitution Check and Required Capabilities are the referencing end; an Amendment applied while the Plan is open enqueues a re-analyze turn.
- **Data Model:** data-model.md cites and declares; the platform reserves, classifies, and at integration transcribes. See [data-model.md](#).
- **Tasks:** tasks.md is part of the bundle and is approved with it.

Ledger — Tasks (tasks.md)

ledger/tasks.md

Tasks are the units of build work. `tasks.md` is authored by the planner as part of the Plan bundle in Spec Kit's format; when the Plan is approved the platform creates one task record per line and owns its lifecycle from there.

Format

Tasks: <feature>

****Input****: plan.md@v2, spec.md@v3

Phase 1 – Setup

- [] T001 Create module skeleton at src/billing/__init__.py
- [] T002 [P] Add migration scaffold migrations/2026_09_subscriptions.sql

Phase 2 – Foundational (blocks all user stories)

- [] T003 Subscription entity and repository in src/billing/models.py

Phase 3 – User Story 1: Start a subscription (P1)

****Goal****: ****Independent test****: ...

- [] T004 [P] [US1] Acceptance tests for US1 scenarios in tests/acceptance/test_subscription.py
- [] T005 [P] [US1] Contract test for POST /subscriptions in tests/contract/test_subscriptions_api.py
- [] T006 [US1] Implement SubscriptionService.start in src/billing/service.py
- [] T007 [US1] Implement POST /subscriptions in src/api/routes/subscriptions.py

****Checkpoint****: US1 independently verifiable and demoable

Phase 4 – User Story 2: Cancel a subscription (P2)

...

Final Phase – Polish

- [] T018 [P] Update quickstart.md validation steps

Dependencies

- T003 blocks T006, T007
 - T004, T005 block T006, T007 (tests first)
- ## Parallel groups
- T004, T005 (different files)

Line grammar: - [] T### [P]? [US#]? <description with an exact file path>. The platform refuses a line with no path, a T### out of sequence, a [US#] naming no story in the spec, or a test task listed after the implementation task it verifies within the same story.

Task kinds

The platform derives each task's **kind** from its phase and description; the planner may state it explicitly with a {kind} suffix.

Kind	Role that claims it	Verifier suite	Notes
setup	implementer	build, lint	Phase 1
foundation	implementer, or implement-data for migrations	build, lint, type-check, unit tests, migration dry-run	Phase 2; blocks every story
test	tester	test files parse and are collected; they fail against the current branch	Story phases; authored from spec and contracts; frozen on completion
implement	implementer (implement, implement-frontend, implement-data)	build, lint, type-check, unit tests, frozen acceptance and contract tests for the story, scanners, write-set check	Story phases
polish	implementer	build, lint, tests	Final phase
bug	implementer · fix	the failing verifier that raised it, plus the full suite	Raised by the platform after a tester triage; no predecessor
defect	planner · defect	analyze gate on the revised bundle	Raised by triage or by a bound; resolves into a new Plan version and new tasks

Statutes

Status	Trigger	Meaning
[PENDING]	Plan approved	Exists; predecessors not all [DONE] or the Plan not yet scheduled
[READY]	Plan [SCHEDULED] and every predecessor [DONE]	On the board; a turn is enqueued for the claiming role
[CLAIMED]	a worker claims the task's turn	Work in progress under a lease
[VERIFYING]	worker signals implementation-complete	The platform runs the verifier suite in a sandbox
[VERIFICATION_FAILED]	verifier run fails	Returned; attempt incremented; a new turn is enqueued with the report, up to MAX_VERIFY_ATTEMPTS
[DONE]	verifier run passes	Successors re-evaluated; test tasks freeze their paths
[ABANDONED]	human decision, or cascade	Terminal; transitive successors cascade

Blocked is a condition: a task with an open blocker is held out of [READY] and reported blocked: true.

Claiming

A task's [READY] transition enqueues a turn for its role. Claiming the turn claims the task; there is no separate task claim signal. When several tasks of one role are ready across features, turns are ordered by the platform's **pickup priority**: feature priority, then completion proximity (done / total tasks of the feature), then unblock count (successors this task releases), then age. [P] tasks of the same story may be claimed concurrently by different workers; the verifier catches a collision as a write-set or merge failure on the feature branch, which is why [P] is only granted to tasks naming different files.

Verification

Implementation-complete carries the branch commit the worker produced. The platform runs the kind's verifier suite against that commit in a sandbox and records a **verifier report** (VerifierRunRecorded): each check, its result, its output. A pass completes the task. A failure returns it with the report as the next turn's input. Nothing a worker or a human can do overrides a failed verifier. See [../platform/verification.md](https://platform/verification.md).

Bugs and Defects

An acceptance or contract test that fails during a later task's verification, or at integration, raises a **triage** turn for the tester. Triage answers one question from the report: does the code diverge from the Plan (→ bug, to the implementer) or does the Plan or Spec have a gap (→ defect, to the planner)? The platform creates the task with the report as its content. Each bug increments the feature's `bug_count`; at `MAX_BUG_RECURRENCE` the platform raises a defect instead and adds a Solution Architect review to the integration gate. A defect resolves into a new Plan version; a version that changes `data-model.md` reopens the data and architecture reviews before its new tasks can be [READY].

Ledger — Data Model

ledger/data-model.md

The Data Model is the project's logical data design as approved and built: one document per entity, per enumeration, and one overview of aggregates and contexts. It is name-keyed, so the ubiquitous language is a database constraint.

Written by: the platform, transcribing an approved Plan's `data-model.md` at integration; or the `reviewer.data` role on an independent change **Read by:** everyone

Two places a data design lives

Where	What	Authored by	Approved by
A Plan's data-model.md	The feature's design: entities cited, entities introduced or revised, as prose and declarations	planner (with the data-model skill)	data reviewer, then the Solution Architect with the Plan
The project Data Model	The system as approved and built	platform transcription at integration	already approved with the Plan

Nothing is written to the project Data Model while a feature is unbuilt. The v6 DATA_MODEL_UPDATE task is replaced by the platform doing the transcription: the declaration was validated, classified, and reviewed at Plan submit; transcription is a comparison, not authorship, so no turn is needed.

The declaration

Each introduced or revised document in `data-model.md` carries a fenced declaration block satisfying the Constitution's representation contract:

```
declaration:
  kind: ENTITY
  name: Subscription
  intent: INTRODUCE          # or REVISE
  aggregate: Customer
  identity: subscription_id (uuid)
  attributes:
    - name: status
      type: SubscriptionStatus
      required: true
    - name: card_last4
      type: string
      sensitivity: PII
  relationships:
    - to: Customer
      cardinality: many-to-one
  lifecycle: [ACTIVE, PAST_DUE, CANCELLED]
```

The platform reads the declaration for four decisions: classification, reservation, whether the security reviewer is required (any sensitivity present), and transcription. Prose without a declaration is refused.

Classification

`change_kind` is computed by diffing the declaration against the current approved document — never asserted:

change_kind	Meaning	At Plan submit	At integration
ADDITIVE	New document, or new optional attributes or relationships	accepted	transcribed
STRUCTURAL	Required attributes, identity, aggregate membership, or lifecycle changed without breaking a citing Plan	accepted; architecture reviewer required	transcribed
BREAKING	Removes or retypes something a live Plan cites	refused , naming the Plans it would break	refused ; a Defect is raised to the planner

A breaking change is a decision to invalidate approved work. It travels the independent path below, where the Solution Architect decides with the impact list, and the feature's Plan then cites the result.

Reservation and citation

- **Reservation:** a (kind, name) introduced by a Plan is reserved from that Plan's submit-for-review. A second Plan introducing the same pair is refused, naming the holder. The reservation lapses on abandonment or on a Plan version that no longer introduces it, and is consumed at transcription.
 - **Citation:** every entity a Plan cites must be [APPROVED]. Citations are recorded and count only while the citing Plan is live, so an abandoned feature never holds an entity against retirement.
-

Statuses (per document)

Status	Meaning
[APPROVED]	In force; Plans may cite it. A feature-driven document enters here directly at transcription
[UNDER_REVIEW]	Independent change awaiting the data reviewer, the architecture reviewer, and, for BREAKING, the Solution Architect
[DEPRECATED]	No new Plan may cite it; existing citations stand and are listed
[RETIRED]	Nothing live cites it

Independent changes

A change with no feature behind it — normalization, an aggregate split, a deprecation, a breaking retype — is proposed by the reviewer. data role as a new document version and reviewed on its own: data and architecture reviewers always, the security reviewer when a sensitivity classification moves, the Solution Architect for BREAKING with the platform-computed impact list. Approval transcribes it immediately.

Storage

```

CREATE TABLE data_model (
    id          SERIAL PRIMARY KEY,
    kind        TEXT NOT NULL CHECK (kind IN ('OVERVIEW','ENTITY','ENUMERATION')),
    name        TEXT NOT NULL,
    current_status TEXT NOT NULL DEFAULT 'APPROVED',
    UNIQUE (kind, name)
);

CREATE TABLE data_model_versions (
    version_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id   INTEGER NOT NULL REFERENCES data_model(id),
    content       TEXT NOT NULL,
    declaration   JSONB NOT NULL,
    change_kind   TEXT NOT NULL CHECK (change_kind IN ('ADDITIVE','STRUCTURAL','BREAKING')),
    feature_id    UUID REFERENCES features(id),          -- NULL on an independent change
    plan_version_id UUID REFERENCES plan_versions(version_id),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT feature_authorization CHECK (
        (feature_id IS NULL AND plan_version_id IS NULL) OR
        (feature_id IS NOT NULL AND plan_version_id IS NOT NULL)
    );

CREATE TABLE data_model_reservations (
    kind        TEXT NOT NULL,
    name        TEXT NOT NULL,
    feature_id  UUID NOT NULL REFERENCES features(id),

```

```
    PRIMARY KEY (kind, name)
);

CREATE TABLE plan_entity_refs (
    feature_id UUID NOT NULL REFERENCES features(id),
    document_id INTEGER NOT NULL REFERENCES data_model(id),
    PRIMARY KEY (feature_id, document_id)
);
```

Ledger — Clarification Session

ledger/clarification.md

A clarification is a bounded, structured question-and-answer session that a role opens against an artifact when it cannot proceed without an answer it does not own. It is not a status, not a task, and not a free dialogue.

Shape

A session belongs to an artifact version and a target:

Field	Meaning
artifact	The Feature Spec, Plan, or Constitution being clarified
asked_by	The role (or human) asking
target	The role or human that answers: Product Owner for intent; analyst for spec meaning; planner for plan meaning; Solution Architect for architecture direction
questions[]	Up to MAX_CLARIFY_QUESTIONS, each with a category from the taxonomy, a recommended answer, and either 2-5 options or a short-answer prompt
answers[]	One per question, recorded as an event when the target answers
applied_version	The artifact version the asker wrote the answers into

Rendered into the artifact:

Clarifications

Session 2026-09-12 (asked by analyst, answered by Product Owner)

- Q: Should a cancelled subscription keep read access until period end? → A: Yes, until period end
- Q: Retention for card_last4 after cancellation? → A: 90 days

Taxonomy

The clarify skill scans against these categories and asks only where the answer would change a requirement, a task, or a verifier: functional scope; domain and data model; interaction and UX flow; non-functional requirements; integrations and external dependencies; edge cases and failure handling; constraints and trade-offs; terminology; completion signals.

Rules

Rule	Enforcement
At most MAX_CLARIFY_QUESTIONS per session	the platform refuses a sixth
At most MAX_CLARIFY_SESSIONS per artifact version lineage; at most MAX_REVIEW_QUESTIONS per reviewer per Plan version	refused at the limit; the asker decides on what it has and records assumptions
Answers are applied, not just logged	the asker's next version must carry the ## Clarifications entries; submit-for-review is refused if a session is answered and unapplied
A session does not change the artifact's status	the artifact reports waiting_on: <target> while a session is open
A session is closed by the asker	when it applies the answers, or when the artifact is abandoned

Who asks whom

Asker	Target	When
analyst	Product Owner	the spec is ambiguous on intent
planner	analyst	the approved spec is ambiguous on meaning; the analyst may in turn open a session with the Product Owner, and the answer flows back through a spec revision (which re-runs the Product Owner's approval only if a requirement changed)
reviewer	planner	the Plan is silent on something the checklist needs (silence is asked about; wrongness is rejected)
architect	Product Owner or Solution Architect	constitution derivation

Humans never open sessions; a human with a question rejects with a reason.

Ledger — Amendment

ledger/amendment.md

An Amendment is a governed change to the Constitution. It merges the v6 Petition (the request) and Amendment (the change record) into one artifact with one lifecycle.

Owner: architect role **Approvers:** Solution Architect, Enterprise Architect

Format

Amendment A-0007: <title>

****Constitution****: 1.2.0 → 1.3.0 (MINOR) · ****Triggered by****: F-0042 (plan@v1, analyze finding C2) | independent

****Status****: [UNDER_REVIEW]

Problem

<what the Constitution prevents or fails to say, with the Plan or evidence that surfaced it>

Proposed change

<the exact Articles and sections changed, as a diff against the current version>

Impact

- Plans not yet integrated citing the affected capability or Article: F-0042, F-0045
- Approved Plans in Build: none

Alternatives considered

Statutes

Status	Trigger	Meaning
[DRAFT]	analyze gate CRITICAL on a constitution gap enqueues an amend turn; or an independent proposal	The architect is authoring
[UNDER_REVIEW]	submit	Both humans hold decisions
[APPROVED]	both approve	The platform applies it in the same transaction: a new Constitution version, version bump per the declared level
[APPLIED]	applied	Terminal; an Amendment authorizes exactly one change
[REJECTED]	either rejects	Returned to the architect; the triggering Plan (if any) is returned to [REJECTED] with the reason, and its Feature Spec may need revision
[ABANDONED]	human decision	Terminal

Effect on open work

Applying an Amendment enqueues a re-analyze turn for every Plan not yet integrated whose Constitution Check or Required Capabilities cite what changed (MINOR), or for every such Plan (MAJOR). A Plan whose re-analyze returns CRITICAL goes to [REJECTED] with the report. A Plan in Build that fails re-analysis is held at the integration gate for the Solution Architect.

The Plan that triggered the Amendment waits at [DRAFT] with `waiting_on: amendment A-0007` and resumes automatically when the Amendment is applied.

Pipeline — Overview

pipeline/overview.md

Six phases, each a declared loop with a gate. Work enters as intent and exits as an integrated, released, monitored change. No phase is skipped; no gate is bypassed.

Intent → [1 Specify] → [2 Plan] → [3 Schedule] → [4 Build] → [5 Integrate & Release] → [6 Operate]

analyst planner platform tester integrator
operator
↑ PO reviewers (Delivery implementer merge queue
anomaly → approves ↑ SA Lead override) verifiers ↑ SA on MEDIUM/HIGH
H new Feature approves

Phase	Loop	Gate out
1 Specify	analyst authors spec.md; clarify sessions with the Product Owner; format verifier	Feature Spec [APPROVED] by the Product Owner
2 Plan	planner authors the bundle; analyze gate; parallel review round; revise	Plan [APPROVED]: analyze clean, every reviewer approved, Solution Architect approved
3 Schedule	platform evaluates priority, predecessors, blockers, WIP, write sets	Plan [SCHEDULED]; tasks with no predecessors [READY]
4 Build	tester and implementer turns per task; sandboxed verifiers; bounded retries; triage	every task [DONE]; no open blocker
5 Integrate & Release	merge queue: rebase, full verifier run, risk band, merge, deploy	DeploymentCompleted healthy

Phase	Loop	Gate out
6 Operate	operator observes; anomalies recorded and routed	an anomaly needing change becomes a new Feature

The platform in every phase

- Enqueues the turn each event calls for and issues the token it runs under
- Validates every signal against the artifact's state and the role's permissions
- Runs every verifier: format checks, the analyze gate, data-model gates, sandboxed suites, the risk score
- Counts every bound and enqueues the declared escalation when one is reached
- Executes every transition atomically with its event
- Derives each feature's phase, `waiting_on`, and `blocked` for humans

The platform does not judge quality beyond what a verifier can check, does not direct how a skill works, and does not resolve an ambiguity: it routes it as a clarification or a blocker.

Gates

Gate	Condition
Specify → Plan	Feature Spec [APPROVED]
Plan → Schedule	Plan [APPROVED] against the current Constitution version
Schedule → Build	Plan [APPROVED]; predecessors [DONE]; no open blocker; WIP below MAX_BUILD_WIP; write set clear under policy
Build → Integrate	every task [DONE] with a passing verifier run; no open blocker; no open Bug or Defect

Gate	Condition
Integrate → Release	rebased branch passes the full suite; risk band LOW, or RiskAccepted
Release → Operate	DeploymentCompleted with health checks green

Detailed phases: [phase-1-specify.md](#), [phase-2-plan.md](#), [phase-3-schedule.md](#), [phase-4-build.md](#), [phase-5-integrate-release.md](#), [phase-6-operate.md](#).

Pipeline — Phase 1: Specify

pipeline/phase-1-specify.md

Purpose: turn intent into an approved Feature Spec. **Roles:** analyst; the Product Owner answers and approves. **Input:** intent captured in the UI. **Output:** spec.md at [APPROVED].

Flow

1. The Product Owner submits intent in the UI: a description, a priority, a type, the measurable constraints the form asks for, and a budget (defaulted from the project). If features in flight are below MAX_OPEN_FEATURES, the platform records FeatureCreated, issues F-####, and enqueues spec.author. Otherwise the intent is queued as PENDING_INTAKE with its position shown to the Product Owner, and admitted when a feature closes — backpressure at the front door rather than a burst of authoring turns competing for the same workers, models, and reviewers.
2. An analyst turn runs the specify skill against the intent, the Charter, and the Constitution, and submits spec.md. The format verifier refuses a version missing a required section.
3. The same turn runs the clarify skill over the draft. If it finds ambiguity that would change a requirement, it opens a session with at most five questions for the Product Owner (ClarificationOpened) and ends the turn; the spec reports waiting_on: product_owner.
4. The Product Owner answers in the UI (ClarificationAnswered); the platform enqueues spec.clarify-apply. The analyst applies the answers into

the spec and ## Clarifications, submits a version, and either opens another session (up to MAX_CLARIFY_SESSIONS) or proceeds.

5. The analyst submits for review. The platform refuses while any [NEEDS CLARIFICATION] remains or a session is answered and unapplied; otherwise SpecSubmittedForReview and the Product Owner's inbox gains the decision.
6. The Product Owner approves (SpecApproved, version pinned, plan.author enqueued) or rejects with reasons (SpecRejected, rejection_count up, spec.author enqueued with the reasons).
7. At MAX_SPEC_REJECTIONS the feature is held with waiting_on: product_owner and a choice: revise the intent (restarts the count) or abandon.

What the analyst owes

- A BUSINESS spec in business language with no mechanism; a TECHNICAL spec with a clear objective and no prescribed solution. Either states every constraint given at intake as an acceptance criterion.
 - User stories ordered P1..Pn, each independently testable, each with Given/When/Then scenarios that a tester can turn into tests without interpretation.
 - FR-### for behavior, SC-### for measurable outcomes, Key Entities without schema, Assumptions for every default taken, Dependencies naming predecessor features.
-

Constraints

- No feature is created before the Constitution is [APPROVED].
- The analyst is the only writer of `spec.md`; the Product Owner changes it only by answering or rejecting.
- The Product Owner's approval pins the version every downstream reader of authorized content uses.

Pipeline — Phase 2: Plan

pipeline/phase-2-plan.md

Purpose: turn an approved Feature Spec into an approved Plan bundle with tasks, in as few turns as the work allows.

Roles: planner; analyzer; reviewer.architecture, reviewer.data, reviewer.security, reviewer.ux; the Solution Architect approves. **Input:** spec.md at [APPROVED].

Output: the Plan bundle at [APPROVED]; task records created.

Flow

1. SpecApproved enqueues plan.author. A planner turn runs the plan skill: it reads the spec, the Constitution, and the Data Model; resolves its technical context (writing research.md for every decision); authors plan.md with the Constitution Check, Required Capabilities, Project Structure, and Write Set; runs the data-model, contracts, threat-model, and quickstart sub-skills to produce the companion files; runs the tasks skill to produce tasks.md; runs the requirements checklist; and submits the bundle as one version. A planner that needs a meaning it cannot resolve opens a clarification session with the analyst rather than guessing.
2. **Deterministic gates** run at submit: format, capabilities, Constitution Check, coverage, data-model citation/reservation/classification, write set, markers. A refusal names the location; the turn fixes and resubmits within the same turn budget.

3. `PlanSubmittedForReview` → `[ANALYZING]`; the platform enqueues `plan.analyze`. The analyzer turn runs the consistency pass and records `PlanAnalyzed`. `CRITICAL` → `[REJECTED]` with the report; `plan.revise` enqueued; `analyze_returns` up (bounded by `MAX_ANALYZE_RETURNS`).
 4. Otherwise `[UNDER_REVIEW]`, agent stage: the platform enqueues one review turn per required reviewer **in parallel** — architecture, data, security always; UX when `tasks.md` has a frontend implementation task. Each reviewer runs its skill against its checklist and the analyze report and returns a verdict (checklist, findings, optional patch). A reviewer may open one bounded session with the planner (`MAX_REVIEW_QUESTIONS`) when the Plan is silent; it rejects when the Plan is wrong.
 5. The stage closes when every reviewer has decided. Any `REJECT` → `PlanRejected` carrying every finding and patch; `[REJECTED]`; `rejection_count` up; `plan.revise` enqueued. All `APPROVE` → `ReviewStageOpened(human)`; the Solution Architect's inbox gains the decision with the bundle, the analyze report, and the verdicts.
 6. The Solution Architect approves (`PlanApproved`, version pinned, `TasksCreated`, reservations confirmed, the feature enters the backlog) or rejects with reasons.
 7. A revision resubmits the whole bundle; reviews on the previous version are invalidated and the agent stage reopens. At `MAX_PLAN_REJECTIONS` the Plan is held for the Solution Architect: redirect (a note the planner revises against, count reset), propose an Amendment (a Constitution gap), or abandon.
-

Turn count

Path	Turns
Clean	planner 1 + analyzer 1 + reviewers 3-4 (parallel) = 5-6, then one human decision
One review round of findings	+ planner 1 + analyzer 1 + reviewers 3-4
Analyze return	+ planner 1 + analyzer 1

The v6 equivalent with three Commissions was roughly thirteen turns and eight serialized hops before a reviewer read anything.

Reviewer checklists

Each reviewer skill ships a checklist; the platform refuses a verdict with an unanswered item.

Reviewer	Checks
architecture	Constitution Check is honest; Required Capabilities are the ones used; module boundaries and patterns conform; write set is complete and minimal; Complexity Tracking justifies every JUSTIFIED
data	every citation resolves; every declaration satisfies the representation contract; classification matches the design; sensitivity is declared on every attribute that needs it; migrations are reversible or say why not

Reviewer	Checks
security	every threat surface in the Constitution has a determination; controls map to the catalog; controlCoverage is complete; contracts expose nothing the spec does not require; dependencies are vetted
ux	every UI story has states, errors, and accessibility per the Constitution's conventions; scenarios are implementable as written

Constraints

- A Plan cannot enter review with an open clarification session, an unmapped requirement, an unknown capability, or a BREAKING declaration.
- Reviewers never reject for brevity; a negative determination is an answer.
- A Constitution gap is an Amendment, never a line in the Plan; the Plan waits at [DRAFT] with `waiting_on: amendment` and resumes on application.
- The Solution Architect is never asked before every machine and agent gate has passed.

Pipeline — Phase 3: Schedule

pipeline/phase-3-schedule.md

Purpose: release approved Plans into Build in the right order without collisions. **Roles:** the platform scheduler; the Delivery Lead may override. **Input:** Plan at [APPROVED]. **Output:** Plan at [SCHEDULED], its branch created, its first tasks [READY].

Why no agent

v6 notified a Scrum Master agent, which read the backlog and signaled a decision the platform then validated against rules it already held. The rules are deterministic. v7 evaluates them directly on every event that can change the answer, and gives a human the override.

Evaluation

Triggered by PlanApproved, FeatureCompleted, FeatureAbandoned, BlockerCleared, IntegrationSucceeded, and a Delivery Lead decision. For each [APPROVED] Plan in priority order:

Condition	Held with
Every predecessor feature [DONE]	PREDECESSOR:F-####
No open blocker on the feature	BLOCKED
Features in Build ([SCHEDULED], [IN_BUILD], [INTEGRATING]) below MAX_BUILD_WIP	WIP

Condition	Held with
Write set does not overlap a feature in Build (policy strict)	WRITE_SET_CONFLICT:F-####
Constitution version at approval is current, or re-analysis passed	REANALYZE

The first Plan meeting every condition is released: PlanScheduled, branch feat/F-####-slug created from the trunk, the approved bundle rendered and committed onto it, tasks with no predecessors → [READY] (each enqueueing its turn). Evaluation continues down the list while WIP allows. Held Plans record PlanHeld with the reason, visible in the backlog.

Ties on priority are broken by key (older first).

Delivery Lead overrides

Decision	Effect
Reorder	sets a scheduling rank above priority for one feature
Force	releases a feature held by WIP or an advisory write-set conflict; never past PREDECESSOR, BLOCKED, or REANALYZE
Hold	keeps a feature out of Build until released

Every override is a recorded decision on the feature stream.

Pipeline — Phase 4: Build

pipeline/phase-4-build.md

Purpose: implement the Plan as tasks, each verified by the platform. **Roles:** tester, implementer; planner for Defects. **Input:** Plan at [SCHEDULED], tasks [READY]. **Output:** every task [DONE]; Plan at [INTEGRATING].

The task loop

TaskReady → turn enqueued for the task's role

- worker claims the turn (= claims the task), materializes the branch
- skill works: reads the bundle under specs/, writes code/tests, commits
- implementation-complete { commit }
- [VERIFYING]: platform runs the kind's suite in a sandbox
 - pass → [DONE]; successors evaluated; test tasks freeze their paths
 - fail → [VERIFICATION_FAILED]; attempt++ ; re-enqueue with the report
at MAX_VERIFY_ATTEMPTS → Defect to the planner with the reports

Tests come first by construction: `tasks.md` lists each story's test tasks before its `implement` tasks and the platform refuses the reverse; the test suite requires the new tests to fail on the branch; the tests are frozen when the task completes; an implementer diff touching them fails.

Roles in the phase

tester (test tasks): authors acceptance tests from the spec's Given/When/Then scenarios and contract tests from `contracts/`, using only those and `quickstart.md`. It never reads implementation, and by ordering there is none to read. On triage turns it reads a verifier report and decides Bug or Defect.

implementer (setup, foundation, implement, polish, bug tasks): implements exactly what the Plan and the task say, within the write set, with unit tests. It reads the spec scenarios because the frozen tests were written from them. It raises a blocker when the Plan is ambiguous rather than deciding; a wrong Plan is a Defect, raised through triage when a test exposes it, or directly by the implementer with evidence.

planner (defect tasks): revises the bundle. A revision that changes `data-model.md` or `contracts/` reopens the data, architecture, and (for sensitivity changes) security reviews before its new tasks are [READY]; any other revision re-runs the analyze gate only. New or changed tasks are added to `tasks.md` with fresh keys; tests for changed scenarios are re-authored by a new test task, which re-freezes them.

Parallelism

Tasks marked [P] in the same phase name different files and may be claimed by different workers at once. Foundational tasks block every story. Stories are independent slices: with capacity, P1's tasks and P2's tasks run concurrently once the foundation is [DONE]. Pickup priority across features: feature priority, completion proximity, unblock count, age.

Bugs and Defects

An acceptance or contract test that passed at its task and fails later (a later task on the branch, or integration) is a regression; the platform enqueues `task.triage`. The tester's verdict raises a bug (implementer, fix skill, the failing check as its verifier) or a defect (planner). Each bug counts toward `MAX_BUG_RECURRENCE`; at the bound the platform raises a defect instead and marks the feature for a Solution Architect review at integration.

Phase gate

When every task is [DONE], no blocker is open, and no Bug or Defect is open, the Plan transitions to [INTEGRATING] and enters the merge queue.

Pipeline — Phase 5: Integrate & Release

pipeline/phase-5-integrate-release.md

Purpose: land the feature on the trunk safely and deploy it.

Roles: integrator; the Solution Architect accepts MEDIUM/HIGH risk; the Delivery Lead clears integration blockers. **Input:** Plan at [INTEGRATING]. **Output:** DeploymentCompleted healthy; Feature [DONE].

Integration

The merge queue processes one feature per repository at a time (see [../platform/concurrency.md](#)):

1. IntegrationStarted records the trunk base. An integrate turn rebases the feature branch onto it, resolving conflicts only within the write set (a conflict outside it is an integration failure), and pushes.
2. The verifier runner executes the full suite on the rebased commit: the whole trunk's tests plus every frozen test of the feature, all scanners, and the risk score. RiskAssessed records the band with its signals.
3. LOW: the platform fast-forwards the trunk. MEDIUM/HIGH, or a feature that reached MAX_BUG_RECURRENCE: the Solution Architect's inbox gains a risk acceptance with the diff, the signals, and the report; the feature steps aside from the lock and re-verifies if the trunk moved before the decision. RiskAccepted merges; a rejection raises a blocker to the Delivery Lead with the Solution Architect's reasons (a

rejected integration usually becomes a Defect).

4. On merge: `DataModelTranscribed` writes the Plan's declarations into the project Data Model (classification recomputed; `BREAKING` here raises a Defect and refuses the merge); `IntegrationSucceeded`; release enqueued.

Failures re-enqueue up to `MAX_INTEGRATION_ATTEMPTS`, then a blocker to the Delivery Lead with the report.

Release

A release turn runs the release skill: it deploys the merged commit through the project's pipeline to the environment the Constitution's Operational Model names, verifies health, and signals `DeploymentCompleted` (→ Feature `[DONE]`, Spec `[DONE]`) or `DeploymentFailed` with the reason (→ blocker to the Delivery Lead; the release skill rolls back per the Operational Model, and the rollback is recorded). Release never re-opens Build: a failed release that needs code is a new Feature or a Defect decided by the Delivery Lead.

Pipeline — Phase 6: Operate

pipeline/phase-6-operate.md

Purpose: observe the live system and route what it finds back into the pipeline. **Roles:** operator; the Delivery Lead and Product Owner decide. **Input:** telemetry, deployment records. **Output:** anomaly records; new Features.

Flow

1. Anomaly rules (error rates, latency baselines from the Constitution's Non-Functional Baselines, health checks) fire; the platform enqueues observe.
 2. An operator turn runs the observe skill: it reads the telemetry window, records an anomaly with severity and evidence (AnomalyRecorded on the platform stream), and links the deployment it followed.
 3. Routing is by rule: CRITICAL → Delivery Lead inbox with a rollback recommendation the release skill can execute on decision; HIGH / MEDIUM → Product Owner inbox as a proposed Feature (intent pre-filled from the evidence); LOW → dashboard only.
 4. A change to the system is a new Feature through Phase 1. The operator never writes code, configuration, or infrastructure.
-

Constraints

- **No self-remediation:** the operator role's only write is an anomaly record.
-

- A rollback is a Delivery Lead decision executed by a release turn and recorded like any deployment.

Platform — Overview

platform/overview.md

The platform owns all state, runs all loops, and is the sole writer to every data store. Workers and humans signal; the platform validates, writes, records, verifies, schedules, and integrates.

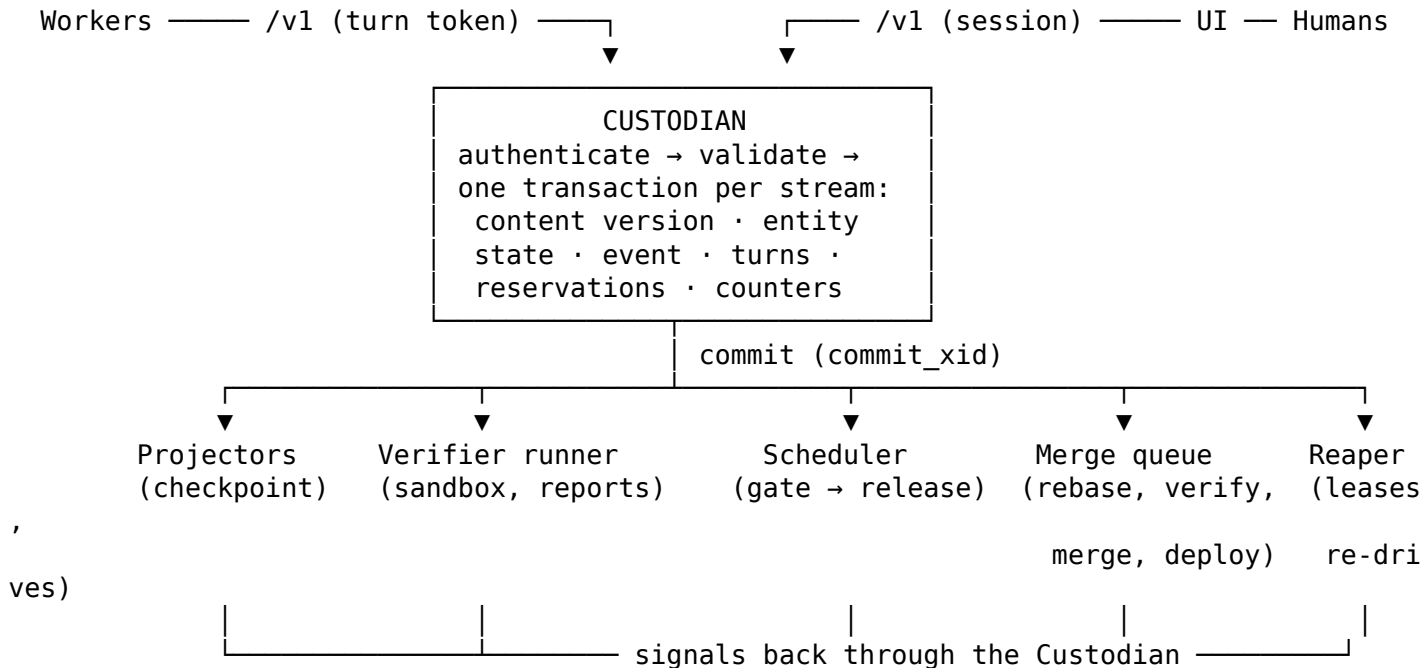
Components

Component	Runs	Owns
Custodian	in the API process, per request	Validation, the one transaction per write, event append, entity update, turn enqueue
Turn queue	a table	The work owed to roles; claims, leases, tokens
Projectors	background, one per read model	Queues, boards, dashboards, inboxes, metrics; a checkpoint each
Verifier runner	background, sandboxed	Verifier suites, analyze deterministic passes, risk scores
Scheduler	background, event-driven	Releasing approved Plans into Build under priority, predecessors, blockers, WIP, and write-set rules
Merge queue	background, one repo at a time	Integrating finished features onto the trunk
Reaper	background, periodic	Requeuing lapsed turns, re-driving interrupted verifier runs
Renderer	on demand	Artifact versions as Markdown and as files in a feature's working tree

Component	Runs	Owns
UI	web	Human intent capture, decisions, answers, inboxes, dashboards, audit

Everything except the Custodian is an asynchronous consumer of the event store or the turn queue. Nothing asynchronous participates in a governing decision: gates read entity state inside the Custodian's transaction.

Architecture



Background components never write to the database directly. They call the Custodian with a platform identity, so their writes are validated and audited like any other.

The Custodian sequence

For every write:

1. **Authenticate**: a turn token (worker) or a session (human). A turn token is honored only while its turn is [CLAIMED] and its lease is live.
2. **Authorize**: the role's permission set against the artifact type, the artifact's stream against the token's scope, and the artifact's current status against the signal.
3. **Validate**: format conformance, gate conditions, bounds, reservations. A refusal returns the reason and writes nothing.
4. **Begin** a transaction; read the stream head.
5. **Write** the content version (if any), the entity state, the event at head + 1, the turns the event calls for, reservations and counters. For a worker call, mark the turn's `write_committed` in the same statement, which is what makes Single Committing Write structural rather than advisory.
6. **Commit**. A unique-index conflict on (`stream_id`, `stream_seq`) means a concurrent write landed on this stream; retry from step 2 up to three times, then refuse with `CONFLICT`.
7. **Return** the result and the commit position (`commit_xid`), which a caller may pass to a read that needs to observe it.

No locks are taken beyond the row-level ones the unique index implies. Two writes to different streams never interact. See [event-store.md](#).

API surface (sketch)

Area	Endpoints	Caller
Turns	POST /v1/turns/claim · POST /v1/turns/{id}/heartbeat (answers CURRENT / SUPERSEDED / CANCELLED with the remaining budget) · POST /v1/turns/{id}/checkpoint · POST /v1/turns/{id}/complete · POST /v1/turns/{id}/fail	workers
Feature Spec	POST /v1/features/{f}/spec/versions · POST /v1/features/{f}/spec/submit · GET /v1/features/{f}/spec[@version]	analyst turn
Plan	POST /v1/features/{f}/plan/versions (the bundle) · POST /v1/features/{f}/plan/submit · GET ...	planner turn
Reviews	POST /v1/features/{f}/plan/reviews (verdict)	reviewer turns
Tasks	GET /v1/features/{f}/tasks · POST /v1/tasks/{t}/implementation-complete	implementer / tester turns

Area	Endpoints	Caller
Clarifications	POST /v1/clarifications · POST /v1/clarifications/{c} /answers · POST /v1/clarifications/{c}/apply	roles, humans
Blockers	POST /v1/blockers · POST /v1/blockers/{b}/clear	roles, Delivery Lead
Decisions	POST /v1/decisions (approve / reject / abandon / schedule override / risk acceptance)	humans
Constitution	POST /v1/constitution/versions · POST /v1/amendments · ...	architect turn, humans
Data Model	GET /v1/data-model · POST /v1/data-model/changes (independent)	all / data reviewer
Reads	GET /v1/features/{f} (aggregate with derived phase, waiting_on, blocked) · projections under /v1/views/* with ?after= cursors and optional ?min_commit=	all

Area	Endpoints	Caller
Admin	worker registry, skill versions, model tiers, stand-downs, projection rebuild, turn retry	Admin, Delivery Lead

Every write accepts an Idempotency-Key; a retried key replays the original result. Every response carries `commit_xid`.

One committing write per turn. Each row above that a worker calls is that turn's single committing write; the Custodian refuses a second on the same token. Checkpoints and heartbeats are exempt because they change no governed state. See [turn-queue.md](#).

Human UI

The UI is a projection consumer plus a decision submitter. It shows each human role an **inbox** of decisions and answers owed, the feature dashboard with derived phase and `waiting_on`, the board, blockers, budgets, and the audit trail; and it submits decisions, clarification answers, intent, and abandonment through the Custodian. Humans never see the turn queue except as "waiting on planner" on a feature, and never see a worker.

Detailed documents

- [turn-queue.md](#) — turns, workers, tokens, leases
 - [event-store.md](#) — streams, optimistic concurrency, partitioning, taxonomy
 - [projections.md](#) — projectors, checkpoints, read-your-writes, rebuild
 - [concurrency.md](#) — isolation by stream, write sets, branches, merge queue
 - [verification.md](#) — verifier suites, analyze gate, risk score
 - [database-schema.md](#) — tables
-

- [recovery.md](#) — what survives a crash and how

Platform — Turn Queue and Workers

platform/turn-queue.md

The turn queue is how the platform gives work to roles. It replaces v6's notification dispatch, outbox, per-agent webhooks, reconciliation sweeps, and claim projection with one table and one protocol.

A turn

Field	Meaning
turn_id	UUID
kind	What the turn is: spec.author, spec.clarify-apply, plan.author, plan.revise, plan.analyze, review.architecture, review.data, review.security, review.ux, task.test, task.implement, task.fix, task.triage, defect.resolve, integrate, release, constitution.author, amend.author, observe, triage-anomaly
role	The role the turn runs as; determines the permission set and the skills allowed
skill, skill_version	The skill the worker must load

Field	Meaning
stream_type, stream_id, feature_id	The scope the token is minted for
trigger	The event (commit_xid) that called for the turn
inputs	Artifact references and reports the skill starts from: versions, verifier report ids, review verdicts, the analyze report
priority, not_before, deadline	Ordering, backoff, and the point at which an unclaimed turn is an alert
attempt, max_attempts	Bound on retries
expected_seconds, slots	Declared by the skill: how long a turn of this kind normally takes, and how much of a worker it occupies. The platform sizes the lease from the first and admission from the second
checkpoint_seq	The last checkpoint the turn stored; a queued attempt starts from it
budget	Token and cost ceiling from the skill, adjustable per project
status	QUEUED · CLAIMED · DONE · FAILED · CANCELLED
worker_id, lease_expires_at, last_heartbeat_at	Liveness
usage, result	Reported by the worker; result is structured per kind

Turns are rows in turns; they are not events. The event that called for a turn is on its stream; the turn's completion signals write further events there. The queue itself is operational

state, rebuildable from the streams if lost (every enqueue rule is a function of an event).

Enqueue rules

The Custodian enqueues turns in the same transaction as the event that calls for them. The rule set is a table, not code paths:

Event	Turn(s) enqueued
FeatureCreated	<code>spec.author</code> (analyst)
ClarificationAnswered	<code>spec.clarify-apply</code> or <code>plan.clarify-apply</code> for the asker
SpecRejected	<code>spec.author</code> with the reasons
SpecApproved	<code>plan.author</code> (planner)
PlanSubmitted (deterministic passes green)	<code>plan.analyze</code> (analyzer)
PlanAnalyzed (no CRITICAL)	one review.* per required reviewer, in parallel
PlanAnalyzed (CRITICAL) or PlanRejected	<code>plan.revise</code> with the report, verdicts, and patches
PlanApproved	none (the scheduler acts)
TaskReady	<code>task.test</code> or <code>task.implement</code> for the task's role
VerifierRunFailed (attempts remain)	<code>task.implement/task.test</code> again with the report
VerifierRunFailed (acceptance or contract test, later task or integration)	<code>task.triage</code> (tester)
BugRaised / DefectRaised	<code>task.fix</code> / <code>defect.resolve</code>
IntegrationStarted	<code>integrate</code>

Event	Turn(s) enqueued
IntegrationSucceeded	release
AmendmentApplied	plan.analyze for every affected open Plan
AnomalyRulesFired	observe

One live turn per (feature_id, kind, artifact). A partial unique index on turns (feature_id, kind, artifact_ref) WHERE status IN ('QUEUED', 'CLAIMED') makes a second enqueue of the same work a no-op. This is the structural form of the v6 "two turns opening the same Story" fix: the queue cannot hold two authoring turns for one artifact.

Supersession cancels. A new artifact version cancels queued review and analyze turns for the previous version; a claimed one keeps running but its completion is refused with SUPERSEDED and recorded as such. Abandonment cancels every queued turn on the feature.

Claiming

```

WITH next AS (
  SELECT turn_id FROM turns
  WHERE status = 'QUEUED'
    AND role = ANY($roles)
    AND not_before <= now()
    AND NOT EXISTS (SELECT 1 FROM skill_standdowns s
                     WHERE s.skill = turns.skill AND s.cleared_at IS NULL)
  ORDER BY priority ASC, created_at ASC
  FOR UPDATE SKIP LOCKED
  LIMIT 1)
UPDATE turns t SET status = 'CLAIMED', worker_id = $worker,
  lease_expires_at = now() + $lease, last_heartbeat_at = now(),
  attempt = attempt + 1
FROM next WHERE t.turn_id = next.turn_id
RETURNING t.*;

```

The response carries the turn and a **turn token**: a signed claim of {turn_id, role, feature_id, stream_id, artifact_scope, exp} with exp at the lease end. Heartbeats extend the lease and reissue the token. The Custodian honors a token only while the turn is [CLAIMED], so a revoked or lapsed turn's token fails on its next write.

priority is computed at enqueue: feature priority, then phase weight (integration and fixes ahead of new authoring, so work in flight finishes before new work starts), then age. A per-role queue cap and MAX_BUILD_WIP bound what enters the queue at all.

One committing write per turn

A turn performs any number of reads and exactly one committing write. The platform refuses a second committing write on the same turn token, naming the write that already landed.

This is the rule v6 most needed and did not have. A v6 spec-authoring turn called create, then draft, then one task-create per task, then one commission request per specialist, then submit — a dozen mutations with no transaction around them, so any failure in the middle left a spec with no version, half a task graph, or a commission the settlement gate would then wait on forever. Every repair for that was a resume path for one more partial state.

The rule is affordable because every artifact in v7 is submitted whole:

Turn kind	Its one write
spec.author	the spec.md version (submit-for-review is part of the same signal when the format gates pass)
plan.author, plan.revise	the whole bundle, tasks.md included, as one version
review.*	the verdict
plan.analyze	the analyze report
task.test, task.implement, task.fix	implementation-complete naming the branch commit
task.triage	the Bug or Defect raise
spec.clarify, review.* asking	the clarification session
*.clarify-apply	the revised version plus the session's application, one signal
integrate	the rebased head
release	the deployment outcome
amend.author	the Amendment version

Work a skill does on the filesystem — commits on a feature branch — is not a committing write: the branch is scratch space until the one signal names a commit. A turn that finds it needs a second write has been mis-specified; it ends and the platform enqueues the next turn.

So a turn ends in exactly two ways: its one committing write — which for some kinds is an opened clarification session rather than a submission — or a failure. Either way it is one atomic, idempotent call, and the artifact is whole on both sides of it.

Checkpoints

A planner turn can run for many minutes and spend a large fraction of a feature's budget. Without checkpoints, a lease lapse or a worker crash discards all of it and the requeued attempt pays for the same tokens again. Wasted model output, not wasted wall-clock, was the expensive part of v6's failures.

```
POST /v1/turns/{id}/checkpoint { label, payload, usage }  
  → { checkpoint_seq }
```

A checkpoint is an intermediate result stored against the turn: a rendered artifact file, a sub-skill's output, a resolved research decision, a diff not yet committed. Checkpoints are **not** events and **not** content versions: they carry no authority, nothing reads them for a decision, and they are deleted when the turn reaches DONE or is cancelled. They exist only so that attempt $n+1$ starts where attempt n stopped.

- Writing a checkpoint extends the lease, so it doubles as a heartbeat.
- A requeued turn is handed `checkpoint_seq` and the stored payloads as additional inputs. The skill decides what to reuse; the platform requires nothing of it.
- Checkpoints are bounded per turn (count and total bytes, from the skill's declaration). Exceeding the bound fails the write, not the turn.
- A checkpoint is the only thing a turn writes that is not its one committing write, and it is exempt precisely because it changes no governed state.

Skills declare where their checkpoints fall. The `plan` skill checkpoints after each sub-skill, so a crash in tasks does not re-derive `data-model.md` and `contracts/`.

Heartbeats say whether the turn is still current

The heartbeat response carries the turn's current standing:

```
POST /v1/turns/{id}/heartbeat { usage }  
  → { status: "CURRENT" | "SUPERSEDED" | "CANCELLED",  
      lease_expires_at, token, budget_remaining }
```

A worker that reads anything but **CURRENT** **stops before its next model call**, reports fail with that reason, and takes the next turn. In v6 a turn whose artifact had moved underneath it discovered this only when its final write was refused, having already paid for every token it spent. Cancellation at the heartbeat bounds that waste to one heartbeat interval.

`budget_remaining` lets a skill shorten its own work — drop an optional section, skip a repair pass — rather than being failed at the ceiling mid-call.

Supersession is the same rule as before, moved earlier: a new artifact version cancels queued review and analyze turns and marks claimed ones **SUPERSEDED**, and abandonment cancels every turn on the feature.

Leases are sized by the skill, not by a global constant

Each skill declares `expected_seconds`. The platform sets the lease to `max(min_lease, lease_factor × expected_seconds)` (default factor 3) and the heartbeat interval to a fraction of it.

A single global lease is wrong in both directions: long enough for a planner turn, it leaves a crashed triage turn stuck for a quarter of an hour; short enough for triage, it reaps planner turns that are working. v6 had one constant, and the heartbeat that was supposed to cover the difference competed for the same connection pool as every board read — so a lease lapse under load meant "the platform is busy", not "the worker died", and the reaper reclaimed live work.

Two rules keep that from recurring:

- **Heartbeats are cheap and independent.** The heartbeat path touches one indexed row, takes no lock beyond that row, and is served from a connection allowance reserved for it. It can never queue behind a projection read or a long report query.
- **A lapse is evidence, not proof.** Requeuing re-validates under the row lock, so a turn that heartbeats between detection and requeue is left alone.

Admission: slots and intake

Two counters bound how much work can be in flight, so that overload is a queue that grows rather than a system that thrashes:

Counter	Bounds	Set by
slots per worker	Concurrent turns one worker runs; each skill declares its slot cost	worker registration + skill
MAX_OPEN_FEATURES	Features not yet [DONE] or [ABANDONED]	project

MAX_OPEN_FEATURES is backpressure at intake. MAX_BUILD_WIP bounds Build, but in v6 nothing bounded the phases before it, so a burst of intent produced a burst of authoring and review turns competing for the same workers, models, and reviewers. Intent submitted past the cap is queued as PENDING_INTAKE with its position, visible to the Product Owner, and admitted as features close.

Completing and failing

complete carries the structured result for the kind (for example a review turn's verdict id, an implement turn's branch commit) and usage. The Custodian records TurnCompleted on the turn's stream and marks the row DONE. A turn that made no write and reports no result is refused: a turn must end in a signal or a failure.

fail carries reason, retryable, and usage. Retryable failures requeue with exponential not_before while attempts remain; then FAILED. A failure classified as **configuration** — the model endpoint refused the credentials (401/402/403) or the model id (404), or the skill version is missing — writes a skill_standdowns row: no turn for that skill is claimable until an Admin clears it, and the queue shows every turn it is holding. One bad model id stands down one skill; every other role keeps working. A content refusal by the model is non-retryable for that turn and surfaces on the artifact as waiting_on: delivery_lead with the reason.

Budget exhaustion fails the turn with BUDGET and is retryable once at the next model tier if the skill allows it.

class says what kind of failure it was, because the responses differ:

Class	Example	Response
TRANSIENT	timeout, rate limit, 5xx	requeue with backoff, resume from checkpoint
CONFIGURATION	rejected key, unknown model id, missing skill version	stand down the skill; an Admin clears it
SUPERSEDED	the artifact moved under the turn	no retry; the replacement turn is already queued
BUDGET	turn or feature ceiling reached	one retry at the next tier, or a blocker to the Product Owner
CONTENT	the model refused the request	no retry; <code>waiting_on:</code> <code>delivery_lead</code> with the reason
INPUT	the turn's inputs do not resolve (a version was abandoned)	no retry; the platform re-derives whether a turn is still owed

Two classes v6 had and v7 does not: a **format** failure and a **parse** failure. Those are handled inside the turn — every model call is validated against the schema the skill declares for that output, and a failure runs the skill's bounded repair pass — so a truncated JSON object or a missing spec section costs one small call instead of a queued turn that will truncate again. A repair pass that does not converge fails the turn as TRANSIENT with the invalid output attached, and the next attempt starts from the last good checkpoint.

Leases and the reaper

A claimed turn whose lease lapses is requeued by the reaper with its attempt count kept and its checkpoints intact, so the next attempt resumes rather than restarts. The worker that held it, if alive, finds its token refused and its next heartbeat CANCELLED, and stops. Re-running a turn is safe because every skill derives its work from the artifact and its checkpoints rather than from anything held in memory, and because every committing write is idempotent by key, so a retry cannot duplicate one.

Worker protocol

```
loop:
  turn ← POST /v1/turns/claim { roles: [...], worker_id, free_slots }
  if none: sleep(backoff); continue
  materialize working tree if skill.needs_fs
  load skill(turn.skill, turn.skill_version)
  run skill with turn.inputs and turn.checkpoints:
    every model call is validated against the output schema the skill
    declares; a parse or format failure runs the skill's bounded repair
    pass rather than failing the turn
    checkpoint after each declared stage
    heartbeat every N seconds with usage; on anything but CURRENT, stop now
  POST /v1/turns/{id}/complete { result, usage }      -- the one committing write
  or POST /v1/turns/{id}/fail { reason, class, retryable, usage }
```

A worker never invents work: it does what the turn says, with the inputs the turn carries. It holds nothing between turns, so two workers are interchangeable and a worker that dies loses at most the work since its last checkpoint.

A worker registers with the Admin surface (id, roles, skills, model tiers) so that the queue can report a turn no registered worker can serve — a QUEUED turn past deadline with no capable worker is an operator alert, not a silent stall.

Observability

Per role and per kind: queue depth, oldest queued age, claim latency, turn duration, attempts per completed turn, checkpoints stored, failure classes, usage and cost. These feed the `turn_metrics` projection and are non-governing, except that a turn past deadline and a skill stand-down are surfaced as conditions on the features they hold.

Who holds the pen. For every artifact with a live turn against it, the `pen_holder` projection reports the turn, its kind and skill version, the worker, the age of its last heartbeat, its checkpoint count, and the budget it has spent. A stuck turn is then a row a human can read and act on — retry it, or let it run — rather than an artifact that has simply stopped moving with nothing to look at. In v6 a claimed task and a crashed agent were indistinguishable until the lease expired.

Platform — Event Store

platform/event-store.md

The event store is the audit trail and the source of truth for projections. v7 changes its shape in three ways: events belong to **streams**, writes use **optimistic concurrency** per stream, and references are **typed columns**.

Schema

```

CREATE TABLE events (
    global_seq    BIGINT GENERATED ALWAYS AS IDENTITY,
    stream_type   TEXT NOT NULL CHECK (stream_type IN ('feature','constitution','data_model','platform')),
    stream_id     UUID NOT NULL,
    stream_seq    INTEGER NOT NULL,
    feature_id    UUID,                                -- denormalized for feature-scoped reads
    type          TEXT NOT NULL,
    payload       JSONB NOT NULL DEFAULT '{}',
    turn_id       UUID,                                -- the turn whose signal produced it, if any
    recorded_by   TEXT NOT NULL DEFAULT 'platform',
    commit_xid    XID8 NOT NULL DEFAULT pg_current_xact_id(),
    recorded_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (global_seq, recorded_at),
    UNIQUE (stream_id, stream_seq, recorded_at)
) PARTITION BY RANGE (recorded_at);

CREATE INDEX ON events (stream_id, stream_seq);
CREATE INDEX ON events (feature_id, global_seq);
CREATE INDEX ON events (type, recorded_at);
CREATE INDEX ON events (commit_xid);

```

Append-only is enforced by BEFORE UPDATE and BEFORE DELETE triggers that raise, as in v6. Monthly partitions keep every index shallow and make archival a partition detach rather than a delete that the trigger would refuse.

Streams

Stream type	One per	Holds
feature	Feature	Everything about the Feature: spec, plan, reviews, tasks, verifier runs, blockers, clarifications, turns' completions, integration, release
constitution	project	Charter, Constitution, Amendments
data_model	project	Independent Data Model changes; transcriptions are recorded on the feature stream and referenced here
platform	project	Scheduler decisions, worker registry changes, stand-downs, projection rebuilds

A stream is the unit of ordering and the unit of concurrency control. Every governed write names exactly one stream.

Optimistic concurrency

```
read  head = max(stream_seq) for stream
validate against entity state (same transaction)
insert event at stream_seq = head + 1
commit
    on unique violation → another writer landed at head + 1: retry from read, ≤ 3 times, then CONFLICT
```

There are no advisory locks. Two writers to different features never touch the same rows. Two writers to the same feature — a reviewer's verdict landing while the planner submits a new version — conflict on the unique index, and the loser re-reads state, at which point its signal may no longer be valid (SUPERSEDED) and is refused with that reason. This is the intended outcome: the conflict is a real one, and the retry resolves it against fresh state.

The entity tables (features, feature_specs, plans, tasks, ...) carry a `stream_seq` column updated with every write, so "the state at event N" is answerable and the gate's read is provably the head it wrote against.

Global ordering for consumers

`global_seq` is monotonic but not gap-safe under concurrent commits: a transaction can take a lower number and commit later. Projectors therefore cursor on `commit_xid`, which is safe by construction:

```
SELECT * FROM events
WHERE commit_xid > $checkpoint
      AND commit_xid < pg_snapshot_xmin(pg_current_snapshot())
ORDER BY commit_xid;
```

Every transaction with an `xid` below the snapshot's `xmin` has finished, so no row can later appear below the watermark. Per-stream order is preserved: a write at `stream_seq` `n+1` read `n` committed, so its `xid` was assigned after `n`'s. `global_seq` remains the human-facing position.

Event taxonomy

Payloads carry version ids and the fields named; artifact identity is in the typed columns.

Feature stream

Event	Payload	Recorded when
FeatureCreated	key, priority, budget, intent	Product Owner submits intent
SpecVersionSubmitted	version_id, turn_id	analyst submits a version
ClarificationOpened / ClarificationAnswered / ClarificationApplied	session_id, target, questions[] / answers[] / version_id	a session moves
SpecSubmittedForReview / SpecApproved / SpecRejected	version_id; reasons[] on rejection	the spec moves
PlanVersionSubmitted	version_id, files[], write_set[], declarations[], citations[], capabilities[]	planner submits the bundle
PlanSubmittedForReview	version_id, gates_passed[]	deterministic gates green
PlanAnalyzed	version_id, report_id, max_severity, coverage	the analyze turn completes
ReviewStageOpened	stage (agent human), reviewers[], version_id	a stage opens
ReviewRecorded	reviewer, decision, verdict_id, version_id	a verdict or human decision
ReviewsInvalidated	version_id, previous_version_id, change_summary	a new version during review
PlanApproved / PlanRejected	version_id; findings[]	the stage closes

Event	Payload	Recorded when
TasksCreated	tasks[] (key, kind, role, paths[], story, predecessors[])	Plan approved
PlanScheduled / PlanHeld	reason (WIP, WRITE_SET_CONFLICT:F-0031, PREDECESSOR:F-0031, BLOCKED)	scheduler decision
TaskReady / TaskClaimed / TaskImplementationComplete	task_id; turn_id; commit	task lifecycle
VerifierRunRecorded	task_id, run_id, commit, checks[], passed	every run
TaskCompleted / TaskVerificationFailed	task_id, run_id, attempt	outcome
PathsFrozen	task_id, paths[]	a test task completes
BugRaised / DefectRaised	task_id, from_run_id, triage_turn_id, evidence_version_id	triage
BugRecurrenceLimitReached	bug_count, defect_task_id	bound
BlockerRaised / BlockerCleared	blocker_id, artifact, reason, resume_context; cleared_by, resolution	blockers

Event	Payload	Recorded when
IntegrationStarted / IntegrationSucceeded / IntegrationFailed	attempt, base_commit; merge_commit, risk; reason	merge queue
RiskAssessed	score, band, signals{}	integration verifier run
RiskAccepted	decided_by, band	Solution Architect decision
DataModelTranscribed	documents[] (kind, name, version_id, change_kind)	integration
DeploymentStarted / DeploymentCompleted / DeploymentFailed	environment, commit; reason	release
FeatureCompleted	—	released and healthy
FeatureAbandoned / TaskAbandoned	reason, decided_by, cascade[]	human decision
TurnEnqueued / TurnCompleted / TurnFailed / TurnCancelled	turn_id, kind, role, skill, skill_version; usage; reason, class	the queue
BudgetExhausted	spent, budget	bound
StatusTransitioned	artifact, from, to, trigger_seq	every status change

Constitution stream

CharterOnboarded, CharterVersionSubmitted,
CharterApproved, ConstitutionVersionSubmitted,
ConstitutionSubmittedForReview, ConstitutionApproved,
ConstitutionRejected, AmendmentProposed,
AmendmentSubmittedForReview, AmendmentApproved,
AmendmentRejected, AmendmentApplied
(constitution_version_id, affected_features[]).

Data Model stream

DataModelChangeProposed, DataModelChangeApproved,
DataModelChangeRejected, DataModelDocumentDeprecated,
DataModelDocumentRetired, plus a reference row
DataModelTranscribed mirroring the feature event.

Platform stream

WorkerRegistered, SkillVersionPublished,
SkillStoodDown, SkillStandDownCleared,
ProjectionRebuilt, SchedulerPolicyChanged.

Reading the audit trail

Feature history is one query:

```
SELECT stream_seq, type, payload, recorded_by, recorded_at  
FROM events WHERE stream_id = $feature ORDER BY stream_seq;
```

The version a decision pinned:

```
SELECT pv.* FROM events e
JOIN plan_versions pv ON pv.version_id = (e.payload->>'version_id')::uuid
WHERE e.stream_id = $feature AND e.type = 'PlanApproved';
```

Cross-feature questions ("every BREAKING refusal this month") are projections, not scans.

Platform — Projections

platform/projections.md

A projection is a read model built asynchronously from the event store by a **projector** with a durable **checkpoint**. Projections serve queues, inboxes, boards, and dashboards. They never sit in the write path and are never consulted for a governing decision.

Why asynchronous

v6 updated every projection inside the Custodian's transaction. Every write paid the cost of every read model it touched, the ready board rewrote a feature's rows on each task event, and a slow projection update lengthened the lock every other writer on that artifact waited for. Under parallel features the write path was the sum of its read models.

In v7 the write path is the stream and the entity row. Read models catch up behind a checkpoint, in batches, on their own schedule. A gate that needs the current state reads the entity row in the same transaction; nothing that decides anything reads a projection.

Projector contract

```
loop:
  batch ← events WHERE commit_xid > checkpoint AND commit_xid < watermark ORDER BY commit_xid LIMIT N
  BEGIN
    for e in batch: apply(e)          -- idempotent per event
    checkpoint ← max(batch.commit_xid)
  COMMIT
  if batch was full: continue immediately
  else: wait for NOTIFY 'events' or poll interval
```

- One row per projector in `projection_checkpoints` (`name`, `commit_xid`, `updated_at`).
- Handlers are idempotent per event so a crash between apply and checkpoint is harmless.
- With replicas, each projector takes `pg_try_advisory_lock(hash(name))`; the others stand by. This is the only advisory lock in the system besides the merge queue's.
- A projector that throws on an event records the event and the error in `projection_errors`, skips it, and continues; the operator sees a lagging, erroring projector rather than a stalled system.

Read-your-writes. Every write returns its `commit_xid`. A read of a projection may pass `?min_commit=<xid>`; the API waits (bounded, default 2 s) for the projector's checkpoint to pass it, then serves the page, or serves it flagged `stale: true` if the bound expires. UI decision flows use this so a human sees their own decision reflected immediately.

Rebuild. Truncate the projection, reset its checkpoint to zero, let it run. `ProjectionRebuilt` is recorded on the platform stream.

Read models

Projection	Serves	Keyed by	Ordering
human_inbox	Each human role's owed decisions and answers: spec approvals, plan approvals, risk acceptances, clarification sessions, held features, budget holds	(human_role, item)	urgency (bound reached, then age)
backlog	[APPROVED] Plans not yet scheduled with the scheduler's current hold reason	feature_id	priority, then key
board	Every task of every feature in Build with status, role, attempt, blocked flag, the turn holding it	task_id	feature priority, then task key
feature_dashboard	One row per feature: derived phase, waiting_on, blocked, budget spent/remaining, bounds consumed, last event	feature_id	last event desc
blockers	Open blockers with the artifact, raiser, reason, resume context, and the human that owns clearing them	blocker_id	age

Projection	Serves	Keyed by	Ordering
review_status	Open review stages: reviewer, decision, verdict id	(feature_id, reviewer)	—
merge_queue_view	Features [INTEGRATING]: position, attempts, risk band, waiting on whom	feature_id	position
pen_holder	Every artifact with a live turn against it: the turn, kind, skill version, worker, heartbeat age, checkpoints stored, budget spent	artifact_ref	heartbeat age desc
turn_metrics	Queue depth, claim latency, duration, attempts, checkpoints, failure classes per role and kind; stand-downs	(role, kind, window)	—
intake_queue_view	Intent queued behind MAX_OPEN_FEATURES, with position	id	position
budget_ledger	Usage and cost per feature, per turn kind, per model tier, against budget	feature_id	—

Projection	Serves	Keyed by	Ordering
data_model_impact	For each document, live citing features; reservations by feature	document_id	—
audit_feature	Rendered timeline per feature	feature_id	stream_seq

The turn queue is a table, not a projection: it is written by the Custodian and read by workers directly.

pen_holder is the answer to "why has this artifact not moved". A live turn with a recent heartbeat is work in progress; one with a stale heartbeat is a worker that died and a lease about to lapse; no turn at all on an artifact that is not terminal and not waiting on a human is a missing enqueue, which is an alert. In v6 these three were indistinguishable — an agent working, an agent crashed, and an agent that never got the notification all looked like an artifact sitting still.

Paging

Every projection read is one bounded page with keyset paging: the ordering ends in a unique column, the page returns an opaque cursor, and a cursor from another queue is refused. Offsets are not used.

Derived phase

A feature's phase is computed by the projector from the Plan and Spec statuses and the tasks, never stored on the entity and never computed by a client:

Phase	Condition
Specify	Spec not [APPROVED]
Plan	Spec [APPROVED], Plan not [APPROVED]
Scheduled	Plan [APPROVED] and not [SCHEDULED]

Phase	Condition
Build	Plan [SCHEDULED] or [IN_BUILD]
Integrate	Plan [INTEGRATING]
Release	integration succeeded, no DeploymentCompleted
Done / Abandoned	terminal

Platform — Concurrency

platform/concurrency.md

How many features can be in flight at once, and where they meet. The design goal is that features contend only where they truly share something, and that each such place has one explicit rule.

Isolation by stream

Every governed write names one stream. Feature streams are independent: optimistic concurrency on (`stream_id`, `stream_seq`) means writers to different features never wait on each other and never retry because of each other. The database-level contention v6 had — advisory locks per artifact, projections updated under those locks, expression-indexed scans of one global table — is gone by construction. See [event-store.md](#).

Cross-stream effects are turns, not writes. An

Amendment that affects twenty open Plans writes one event on the constitution stream and enqueues twenty `plan.analyze` turns; each turn later writes to its own feature stream. A BREAKING Data Model change resolves its impact list from `plan_entity_refs` and raises Defects the same way. No transaction ever spans two streams.

Where features actually meet

Shared thing	Conflict	Rule	Enforced at
Data Model names	Two features introduce Subscription	Reservation to the first Plan past submit-for-review; the second is refused naming the holder	Plan submit
Data Model shape	A feature revises an entity another live Plan cites	change_kind computed; BREAKING refused at submit and at integration; goes through the independent review with the impact list	Plan submit, integration
The Constitution	An Amendment changes what open Plans relied on	Re-analyze turns for affected Plans; CRITICAL returns them	Amendment applied
The codebase	Two features change the same files	Write sets, branches, and the merge queue (below)	Scheduling, verification, integration
Human attention	Too many decisions owed	MAX_BUILD_WIP and per-role queue caps; inbox ordered by urgency	Scheduling, enqueue
Workers	More turns than capacity	Queue depth is the metric; add workers	—

Write sets

Every Plan declares in `plan.md`:

```
## Write Set
- src/billing/**
- src/api/routes/subscriptions.py
- migrations/2026_09_*.sql
```

The platform normalizes the globs and stores them on the Plan. Two uses:

- **Scheduling.** The scheduler computes overlap between a candidate Plan's write set and every Plan in Build. Under policy `strict` (default) an overlapping candidate is held with `WRITE_SET_CONFLICT:F-0031` until the other integrates; under `advisory` it is scheduled after the other in priority order and the conflict is left to the merge queue. Foundational paths a project marks shared (a router registry, a migration index) are excluded from overlap and left to rebase.
 - **Verification.** An implementer diff outside the write set fails the write-set check unless the task's line in `tasks.md` names the path. A Plan that needs a wider set is revised, which is a visible decision rather than a silent expansion.
-

Branches and the working tree

At scheduling the platform creates `feat/F-####-slug` from the trunk head and commits the rendered, approved bundle under `specs/F-####-slug/`. Every implementer and tester turn works on that branch; the commit its implementation-complete names is what the verifier runs. Tasks marked [P] may run concurrently on the same branch because they name different files; a collision surfaces as a verifier failure on the second to complete, and the returned turn rebases and retries.

The trunk is never written by a worker. Only the merge queue advances it.

The merge queue

A feature whose tasks are all [DONE] enters [INTEGRATING]. The queue is ordered by feature priority, then by entry time, and processes one feature per repository at a time under a repository advisory lock — the single point of serialization in the system, placed where serialization is unavoidable and cheap.

```
take head feature F
IntegrationStarted(attempt n, base = trunk head)
enqueue integrate turn:
    rebase feat/F onto trunk (or merge trunk in, per repo policy)
    resolve conflicts within the write set; a conflict outside it is a failure
    push the rebased branch
verifier runner: full suite on the rebased commit (build, lint, types, all
unit tests, all frozen acceptance and contract tests of F and of the trunk,
scanners) + risk score
fail → IntegrationFailed; attempt < MAX_INTEGRATION_ATTEMPTS ? re-enqueue
    : blocker to the Delivery Lead with the report
pass → RiskAssessed; band LOW → merge; MEDIUM/HIGH → wait for RiskAccepted
platform fast-forwards trunk to the verified commit; DataModelTranscribed;
IntegrationSucceeded; release turn enqueued
release lock; take next
```

A feature waiting on a Solution Architect's risk decision does not hold the lock: it steps aside, and re-verifies against the new trunk head when the decision arrives if the trunk moved (a second run is cheap; a stale merge is not).

Multi-repository projects run one queue per repository; a feature whose write set spans repositories integrates them in the order its Plan declares, and a failure in the second rolls the first back by revert.

Work in progress

Limit	Bounds	Why
MAX_BUILD_WIP	Plans at [SCHEDULED], [IN_BUILD], [INTEGRATING]	Reviewer and human attention, merge-queue depth
per-role queue cap	QUEUED turns per role	Workers per role; keeps claim latency bounded and visible
FEATURE_BUDGET	spend per feature	The loop's stop

Held features are visible in the backlog with the reason and the feature that holds them.

Capacity model

Throughput is bounded by the slowest of: worker capacity per role (add workers), the merge queue's serial integration time (keep verifier suites fast; shard by repository), and human decision latency (bounded by WIP, not by the platform). Nothing in the database is a shared bottleneck between features except the merge queue's lock and the projectors' single-writer loops, both of which are off the write path.

Platform — Verification

platform/verification.md

A worker's claim of success never completes anything. The platform runs the verifier, records the report, and decides. This document defines the verifier suites, the analyze gate, frozen paths, and the risk score.

Principle

In a loop the verifier is the bottleneck, and the generator's opinion of its own work is not a verifier. v6 asked each Guild Lead to score its own change on a rubric and combined the scores into a risk band; v7 replaces self-assessment with deterministic checks the platform runs in a sandbox and a risk score computed from signals those checks produce.

Project verifier configuration

A project declares its verifiers once, in the repository:

```
# .aeneas/verify.yml
image: ghcr.io/org/project-ci:2026.09
commands:
  build:      make build
  lint:       make lint
  typecheck:  make typecheck
  unit:       make test-unit
  acceptance: make test-acceptance ARGS="$PATHS"
  contract:   make test-contract ARGS="$PATHS"
  migration:  make migrate-dry-run
scanners:
  - static_analysis
  - dependency_audit
```

- secrets
- container

```
timeouts: { default: 900 }
```

The verifier runner executes suites in a fresh container from image on the named commit, with no network except what the project allows, and captures every command's exit code and output.

Suites by task kind

Kind	Checks
setup	build, lint
foundation	build, lint, typecheck, unit, migration
test	test files parse and are collected; the story's acceptance and contract tests fail against the branch (a test that passes before implementation tests nothing); no production path touched
implement	build, lint, typecheck, unit, acceptance and contract tests for the task's story (frozen), scanners, write-set check, frozen-path check
polish	build, lint, typecheck, unit, all frozen tests of the feature
bug	the check that raised it, then the implement suite
integration	everything above across the whole feature and the trunk, plus risk assessment

Every run is recorded as `VerifierRunRecorded` with each check's result and output. A failing run returns the task with the report as the next turn's input; nothing overrides it. A run that fails for infrastructure reasons (the image would not start, a timeout before any check ran) is recorded as `INFRA` and re-driven by the reaper, not counted as an attempt.

Frozen paths

When a test task completes, the platform records `PathsFrozen` for the test files it produced. From then on, any implementer diff on the feature branch that changes a frozen path fails the frozen-path check. The tester may revise its own tests only through a new test task raised by a Defect resolution (the Plan changed, so the tests may). This is Test Independence made structural.

Checklist and quickstart assertions

`quickstart.md` steps and checklists/ items may be written as machine assertions:

- [] `POST /subscriptions` with a valid body returns 201 and a `subscription\_id` {assert: contract:subscriptions#start}
- [] Cancelling keeps read access until `current\_period\_end` {assert: acceptance:US2-S3}
- [] No endpoint returns `card\_number` {assert: secrets-in-responses}

An assertion names a check the runner can evaluate; an item without one is a reviewer's item and is answered in the verdict. Coverage at Plan submit requires every SC-### to be reachable by at least one assertion or one reviewer checklist item.

The analyze gate

Runs when a Plan is submitted, before any reviewer turn.

Deterministic passes (platform):

Pass	Refuses when
Format	any bundle file misses a required section or numbering
Capabilities	## Required Capabilities names an identifier the Constitution does not grant
Constitution Check	any Article marked FAIL, or an Article missing from the table
Coverage	an FR-###, user story, or SC-### maps to nothing; a T### maps to no story or requirement
Data Model	a citation of an unknown/deprecated/retired document; a reservation conflict; a BREAKING classification; a declaration failing the representation contract
Write set	empty, or a path outside the repository
Markers	any NEEDS CLARIFICATION left in the bundle

Consistency pass (an analyzer turn): duplication, ambiguity, underspecification, constitution-alignment reasoning, cross-artifact inconsistency (terminology drift, ordering contradictions between `plan.md` and `tasks.md`). Output is a report table with severity; CRITICAL returns the Plan to the planner; lower severities travel to the reviewers as context. Reaching MAX_ANALYZE_RETURNS holds the Plan for the Solution Architect with the report rather than re-running the planner again.

Risk score

Computed by the platform at integration from the integration run and the Plan:

Signal	Source	Contribution
Diff size	lines added + deleted, log-scaled	0-2
Spread	distinct top-level modules touched	0-2
Surface	contracts added, changed, or removed (contracts/ diff against trunk)	0-3; a removed or changed existing contract is ≥ 2
Data	max change_kind among declarations (ADDITIVE 0, STRUCTURAL 2); any sensitivity introduced +1	0-3
Dependencies	new or upgraded dependencies; any with known advisories	0-3; any advisory ≥ 2
Findings	scanner findings by severity; HIGH/CRITICAL are blockers, not signals	0-2
Tests	ratio of frozen tests to FR-###; acceptance tests that changed since freezing (should be 0)	0-2
History	bug_count, verify attempts, integration attempts for the feature	0-3

score = Σ contributions (0–20). Bands: LOW < 6, MEDIUM 6–11, HIGH $\geq$ 12, project-tunable. LOW merges on the verifier's pass alone; MEDIUM and HIGH wait for RiskAccepted from the Solution Architect, who sees the signals, the report, and the diff. A Solution Architect review is also required, regardless of band, when MAX_BUG_RECURRENCE was reached.

The score is recorded (RiskAssessed) with its signals so that the band a human accepted is auditable and the weights can be tuned against outcomes.

Platform — Database Schema

platform/database-schema.md

PostgreSQL. Five groups: entity tables (aggregate state), version tables (append-only content), the event store, operational tables (turns, verifier runs, reservations, frozen paths, blockers, budgets), and projections. Projections are in [projections.md](#); the event store in [event-store.md](#).

Sequences

```
CREATE SEQUENCE feat_seq START 1;    -- F-####  
CREATE SEQUENCE amnd_seq START 1;    -- A-####
```

Tasks are numbered per feature (T###) from `tasks.md`; the platform assigns `task_key` at `TasksCreated`.

Entity tables

```

CREATE TABLE features (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sequence_number    INTEGER NOT NULL UNIQUE DEFAULT nextval('feat_seq'),
    slug              TEXT NOT NULL,
    priority           INTEGER NOT NULL CHECK (priority BETWEEN 1 AND 5),
    feature_type       TEXT NOT NULL CHECK (feature_type IN ('BUSINESS','TECHNICAL')),
    intent            TEXT NOT NULL,
    budget_tokens      BIGINT NOT NULL,
    budget_micros      BIGINT NOT NULL,
    branch            TEXT,
    current_status     TEXT NOT NULL DEFAULT 'ACTIVE',    -- ACTIVE | DONE | ABANDONED
    stream_seq         INTEGER NOT NULL DEFAULT 0,
    created_by         TEXT NOT NULL,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE feature_specs (
    feature_id         UUID PRIMARY KEY REFERENCES features(id),
    current_status     TEXT NOT NULL DEFAULT 'DRAFT',
    approved_version_id UUID,
    rejection_count    INTEGER NOT NULL DEFAULT 0,
    clarify_sessions   INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE plans (
    feature_id         UUID PRIMARY KEY REFERENCES features(id),
    current_status     TEXT NOT NULL DEFAULT 'DRAFT',
    approved_version_id UUID,

```

```

    constitution_version_id UUID,          -- the version the approval was checked against
    rejection_count          INTEGER NOT NULL DEFAULT 0,
    analyze_returns          INTEGER NOT NULL DEFAULT 0,
    bug_count                INTEGER NOT NULL DEFAULT 0,
    integration_attempts     INTEGER NOT NULL DEFAULT 0,
    write_set                TEXT[] NOT NULL DEFAULT '{}',
    risk_band                TEXT,
    scheduled_at             TIMESTAMPTZ
);

CREATE TABLE tasks (
    id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    feature_id              UUID NOT NULL REFERENCES features(id),
    task_key                TEXT NOT NULL,          -- T017
    kind                    TEXT NOT NULL CHECK (kind IN ('setup','foundation','test','implementation','polish','bug','defect')),
    role                    TEXT NOT NULL,
    story                   TEXT,                  -- US1
    description             TEXT NOT NULL,
    paths                   TEXT[] NOT NULL DEFAULT '{}',
    parallel                BOOLEAN NOT NULL DEFAULT false,
    current_status          TEXT NOT NULL DEFAULT 'PENDING',
    attempt                 INTEGER NOT NULL DEFAULT 0,
    UNIQUE (feature_id, task_key)
);

CREATE TABLE task_dependencies (

```

```

    predecessor_id UUID NOT NULL REFERENCES tasks(id),
    successor_id   UUID NOT NULL REFERENCES tasks(id),
    PRIMARY KEY (predecessor_id, successor_id),
    CHECK (predecessor_id <> successor_id)
);

CREATE TABLE feature_dependencies (
    predecessor_id UUID NOT NULL REFERENCES features(id),
    successor_id   UUID NOT NULL REFERENCES features(id),
    PRIMARY KEY (predecessor_id, successor_id)
);

CREATE TABLE constitution_meta (
    id                UUID PRIMARY KEY,
    current_status    TEXT NOT NULL DEFAULT 'DRAFT',
    approved_version_id UUID,
    semver            TEXT,
    stream_seq        INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE amendments (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sequence_number    INTEGER NOT NULL UNIQUE DEFAULT nextval('amnd_seq'),
    level             TEXT NOT NULL CHECK (level IN ('MAJOR', 'MINOR', 'PATCH')),
    triggered_by       UUID REFERENCES features(id),
    current_status     TEXT NOT NULL DEFAULT 'DRAFT',
    applied_constitution_version_id UUID
);

```

`data_model`, `data_model_versions`,
`data_model_reservations`, and `plan_entity_refs` are in
[../ledger/data-model.md](#).

The heartbeat path (`UPDATE` turns `SET lease_expires_at`,
`last_heartbeat_at` `WHERE turn_id = $1 AND worker_id =`
`$2`) touches one row by primary key and takes no other lock. It
is served from a reserved connection allowance so it can never
queue behind a projection read — a lapsed lease must mean a
dead worker, never a busy platform.

Version tables

All append-only with raise-on-mutation triggers.

```

CREATE TABLE spec_versions (
    version_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    feature_id  UUID NOT NULL REFERENCES features(id),
    content     TEXT NOT NULL,
    parsed      JSONB NOT NULL,          -- stories, FRs, SCs, markers, clarifications
    turn_id     UUID,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE plan_versions (
    version_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    feature_id  UUID NOT NULL REFERENCES features(id),
    files       JSONB NOT NULL,          -- { "plan.md": "...", "research.md": "...", "con
tracts/openapi.yaml": "...", ... }
    parsed      JSONB NOT NULL,          -- capabilities, write set, declarations, citatio
ns, tasks, coverage map
    turn_id     UUID,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE review_verdicts (
    verdict_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    feature_id  UUID NOT NULL REFERENCES features(id),
    plan_version_id  UUID NOT NULL REFERENCES plan_versions(version_id),
    reviewer    TEXT NOT NULL,
    decision     TEXT NOT NULL CHECK (decision IN ('APPROVE','REJECT')),
    content     TEXT NOT NULL,
    checklist    JSONB NOT NULL,

```

```

    findings          JSONB NOT NULL,
    patch             TEXT,
    turn_id           UUID,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE analyze_reports (
    report_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    feature_id        UUID NOT NULL REFERENCES features(id),
    plan_version_id   UUID NOT NULL REFERENCES plan_versions(version_id),
    findings          JSONB NOT NULL,
    coverage          JSONB NOT NULL,
    max_severity      TEXT NOT NULL,
    turn_id           UUID,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE constitution_versions (
    version_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    content            TEXT NOT NULL,
    parsed            JSONB NOT NULL,          -- articles, capability map, control catalog, rep
    resentation contract
    semver            TEXT NOT NULL,
    amendment_id      UUID REFERENCES amendments(id),
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE amendment_versions ( ... );

```

```
CREATE TABLE charter_versions ( ... );  
CREATE TABLE task_evidence_versions ( ... ); -- Bug/Defect evidence, verifier reports as  
content
```

Operational tables

```

CREATE TABLE turns (
    turn_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    kind             TEXT NOT NULL,
    role            TEXT NOT NULL,
    skill           TEXT NOT NULL,
    skill_version    TEXT NOT NULL,
    stream_type      TEXT NOT NULL,
    stream_id        UUID NOT NULL,
    feature_id       UUID REFERENCES features(id),
    artifact_ref     TEXT NOT NULL,
    trigger_xid      XID8 NOT NULL,
    inputs           JSONB NOT NULL DEFAULT '{}',
    priority         INTEGER NOT NULL,
    not_before       TIMESTAMPTZ NOT NULL DEFAULT now(),
    deadline         TIMESTAMPTZ,
    attempt          INTEGER NOT NULL DEFAULT 0,
    max_attempts     INTEGER NOT NULL DEFAULT 3,
    expected_seconds INTEGER NOT NULL,           -- from the skill; sizes the lease
    slots            INTEGER NOT NULL DEFAULT 1, -- worker capacity this turn occupies
    checkpoint_seq   INTEGER NOT NULL DEFAULT 0,
    write_committed  BOOLEAN NOT NULL DEFAULT false, -- Single Committing Write
    budget_tokens    BIGINT NOT NULL,
    status           TEXT NOT NULL DEFAULT 'QUEUED' CHECK (status IN ('QUEUED','CLAIMED', 'DONE','FAILED','CANCELLED')),
    worker_id        TEXT,
    lease_expires_at TIMESTAMPTZ,
    last_heartbeat_at TIMESTAMPTZ,
    usage            JSONB NOT NULL DEFAULT '{}',

```

```

    result            JSONB,
    failure           JSONB,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON turns (role, priority, created_at) WHERE status = 'QUEUED';
CREATE INDEX ON turns (lease_expires_at) WHERE status = 'CLAIMED';
CREATE UNIQUE INDEX turns_one_live
    ON turns (stream_id, kind, artifact_ref) WHERE status IN ('QUEUED','CLAIMED');

CREATE TABLE turn_checkpoints (
    turn_id           UUID NOT NULL REFERENCES turns(turn_id) ON DELETE CASCADE,
    seq               INTEGER NOT NULL,
    label             TEXT NOT NULL,
    payload           JSONB NOT NULL,
    bytes             INTEGER NOT NULL,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (turn_id, seq)
);
-- Not events and not content versions: no authority, nothing reads them for a
-- decision, deleted with the turn. They exist so a requeued attempt resumes.

CREATE TABLE skill_registry (
    skill             TEXT NOT NULL,
    version           TEXT NOT NULL,
    role             TEXT NOT NULL,
    kinds            TEXT[] NOT NULL,
    model_tier        TEXT NOT NULL,
    expected_seconds  INTEGER NOT NULL,

```

```

    slots            INTEGER NOT NULL DEFAULT 1,
    max_checkpoints  INTEGER NOT NULL DEFAULT 16,
    max_checkpoint_bytes INTEGER NOT NULL DEFAULT 4194304,
    output_schemas  JSONB NOT NULL DEFAULT '{}',    -- per output, for in-turn validation
on
    budget_tokens    BIGINT NOT NULL,
    published_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (skill, version)
);

CREATE TABLE workers (
    worker_id        TEXT PRIMARY KEY,
    roles            TEXT[] NOT NULL,
    skills           JSONB NOT NULL,                -- skill → versions available
    total_slots      INTEGER NOT NULL,
    last_seen_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE skill_standdowns (
    skill            TEXT NOT NULL,
    reason           TEXT NOT NULL,
    stood_down_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    cleared_at       TIMESTAMPTZ,
    cleared_by       TEXT
);

CREATE TABLE verifier_runs (
    run_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

feature_id  UUID NOT NULL REFERENCES features(id),
task_id     UUID REFERENCES tasks(id),           -- NULL for an integration run
commit      TEXT NOT NULL,
suite       TEXT NOT NULL,
status      TEXT NOT NULL CHECK (status IN ('QUEUED','RUNNING','PASSED','FAILED','IN
FRA')),
checks      JSONB NOT NULL DEFAULT '[]',
risk        JSONB,
started_at  TIMESTAMPTZ, finished_at TIMESTAMPTZ
);

CREATE TABLE frozen_paths (
feature_id  UUID NOT NULL REFERENCES features(id),
path        TEXT NOT NULL,
task_id     UUID NOT NULL REFERENCES tasks(id),
PRIMARY KEY (feature_id, path)
);

CREATE TABLE blockers (
id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
feature_id  UUID REFERENCES features(id),
artifact_ref TEXT NOT NULL,
raised_by   TEXT NOT NULL,
reason      TEXT NOT NULL,
resume_context JSONB,
owner_role  TEXT NOT NULL,                      -- the human role that clears it
cleared_at  TIMESTAMPTZ, cleared_by TEXT, resolution TEXT
);

```

```

CREATE TABLE clarification_sessions (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    feature_id    UUID REFERENCES features(id),
    artifact_ref  TEXT NOT NULL,
    version_id    UUID NOT NULL,
    asked_by     TEXT NOT NULL,
    target       TEXT NOT NULL,
    questions    JSONB NOT NULL,
    answers      JSONB,
    applied_version_id UUID,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE decisions (                                -- human sign-offs and acceptanc
es
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    feature_id    UUID REFERENCES features(id),
    artifact_ref  TEXT NOT NULL,
    version_id    UUID,
    human_role    TEXT NOT NULL,
    human_id     TEXT NOT NULL,
    decision      TEXT NOT NULL,
    reasons      JSONB,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE usage_ledger (
    id            BIGSERIAL PRIMARY KEY,

```

```

feature_id    UUID REFERENCES features(id),
turn_id      UUID NOT NULL REFERENCES turns(turn_id),
model        TEXT NOT NULL,
tier         TEXT NOT NULL,
prompt_tokens BIGINT, completion_tokens BIGINT, cached_tokens BIGINT,
calls INTEGER, duration_ms BIGINT,
recorded_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE model_prices ( engine TEXT, model TEXT, effective_from TIMESTAMPTZ, input_m
icros BIGINT, output_micros BIGINT, cached_micros BIGINT, PRIMARY KEY (engine, model, ef
fective_from) );

CREATE TABLE intake_queue (                                     -- MAX_OPEN_FEATURES backpressu
re
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    intent      TEXT NOT NULL,
    priority    INTEGER NOT NULL,
    submitted_by TEXT NOT NULL,
    submitted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    admitted_feature_id UUID REFERENCES features(id)           -- NULL while PENDING_INTAKE
);

CREATE TABLE processed_requests (
    idempotency_key TEXT PRIMARY KEY,
    result_snapshot JSONB NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```
CREATE TABLE projection_checkpoints (  
    name          TEXT PRIMARY KEY,  
    commit_xid    XID8 NOT NULL DEFAULT '0',  
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

What is gone from v6

Commission tables and refs; spec_review_rounds; the scanning-failure projection; notification_outbox; projection_active_claims; agent_webhooks; archetype per-subtype rows and archetype_meta; petitions; story_dependencies and spec_dependencies (features depend on features); DATA_MODEL_UPDATE and AGGREGATE_DELTA task types; the changeDelta payloads.

Platform — Recovery

platform/recovery.md

What survives a crash, and how. The mechanisms are fewer than in v6 because the turn queue replaced notifications, sweeps, and claim projections with one leased table.

Failure	What is at risk	Recovery
Worker dies mid-turn	The model output since its last checkpoint	Lease lapses; the reaper requeues the turn with its attempt count and its checkpoints. The next worker resumes from the last checkpoint rather than re-deriving. The dead worker's token is refused, and its next heartbeat (if it revives) answers CANCELLED
Worker alive but the artifact moved	Every token the turn spends after that	The heartbeat answers SUPERSEDED; the worker stops before its next model call. Its completion would be refused anyway, so the only thing at stake is waste, bounded to one heartbeat interval
A model call returns unparseable or malformed output	The turn	The skill's bounded repair pass runs inside the turn against the declared output schema. Only a non-converging repair fails the turn, and it resumes from the last checkpoint

Failure	What is at risk	Recovery
Lease lapses because the platform was slow, not because the worker died	Live work reclaimed	Cannot happen by construction: the heartbeat path is a single indexed row update on a reserved connection allowance, and requeuing re-validates under the row lock, so a turn that heartbeated between detection and requeue is left alone
A turn is enqueued twice	Two turns authoring one artifact	Cannot happen: a partial unique index allows one live turn per (stream, kind, artifact); the second enqueue is a no-op
A turn dies between two writes	A half-created artifact	Cannot happen: a turn has exactly one committing write
Platform dies mid-write	Nothing: the transaction rolled back	The caller retries with the same Idempotency-Key
Platform dies after commit, before the response	A duplicate on retry	processed_requests replays the stored result
Verifier runner dies mid-run	A run stuck at RUNNING	The reaper re-drives runs past their timeout as INFRA (not counted as an attempt); suites are deterministic and re-runnable

Failure	What is at risk	Recovery
Projector dies	Nothing: the checkpoint advances with the batch	It resumes from the checkpoint; handlers are idempotent
Merge queue dies mid-integration	The repository lock, a half-pushed branch	The lock is session-scoped and drops; the next pass finds the [INTEGRATING] feature with IntegrationStarted and no outcome, re-enqueues the integrate turn, which rebases again from the recorded base or the new trunk head
Model endpoint refuses the configuration	Every turn of that skill	skill_standdowns holds the skill's turns visibly; an Admin fix plus clear resumes them at once
Model refuses one request	That turn	FAILED non-retryable; the feature shows waiting_on: delivery_lead; retry by decision
No worker can serve a role	Turns age past deadline	The queue reports it as an operator alert on the features it holds
The turn queue table is lost	Owed work	Rebuildable: replay the enqueue rules over each stream from the last completed turn per (stream, kind, artifact)

Startup

On boot the platform runs one pass: requeue lapsed turns, re-drive timed-out verifier runs, resume the merge queue, start projectors from their checkpoints. Nothing waits for a periodic sweep to notice; the periodic reaper exists for the same three conditions arising while the platform is up.

Detection is state-based

Every recovery condition is a direct query: turns WHERE status='CLAIMED' AND lease_expires_at < now(), verifier_runs WHERE status='RUNNING' AND started_at < now() - timeout, plans WHERE current_status='INTEGRATING' with no live integrate turn. No consumer cursor over the event store is involved except the projectors' own.

Idempotency

Every write accepts an Idempotency-Key. The Custodian claims the key in the same transaction as the write; a retry replays the stored result; a concurrent duplicate blocks on the key row until the first commits, then replays. Workers generate one key per logical write and retry transient failures with backoff.

Workflows — Main Workflow

workflows/main-workflow.md

The happy path from intent to a live, monitored change, with the turns and decisions it costs.

Step	Actor	Turn / decision	Events
1	Product Owner	submits intent in the UI	FeatureCreated
2	analyst turn	specify + clarify; opens a session if needed	SpecVersionSubmitted, ClarificationOpened
3	Product Owner	answers the session	ClarificationAnswered
4	analyst turn	applies answers; submits for review	SpecVersionSubmitted, ClarificationApplied, SpecSubmittedForReview
5	Product Owner	approves	SpecApproved
6	planner turn	plan + companions + tasks; submits the bundle	PlanVersionSubmitted, PlanSubmittedForReview
7	analyzer turn	consistency pass	PlanAnalyzed
8	reviewer turns × 3-4, parallel	verdicts	ReviewStageOpened(agent), ReviewRecorded × n
9	Solution Architect	approves	ReviewStageOpened(human), ReviewRecorded, PlanApproved, TasksCreated
10	scheduler	releases; branch created	PlanScheduled, TaskReady × k
11	tester / implementer turns	per task; verifiers run per task	TaskClaimed, TaskImplementationComplete, VerifierRunRecorded, TaskCompleted, PathsFrozen

Step	Actor	Turn / decision	Events
12	merge queue + integrator turn	rebase, full suite, risk	IntegrationStarted, RiskAssessed, (RiskAccepted), DataModelTranscribed, IntegrationSucceeded
13	integrator turn	deploy	DeploymentStarted, DeploymentCompleted, FeatureCompleted
14	operator turns	observe	AnomalyRecorded as needed

Human decisions on the happy path: three (spec, plan, and, for MEDIUM/HIGH risk, integration). Model turns before Build: five to seven.

Review model

A review stage opens with `ReviewStageOpened` naming its reviewers; each records one decision; the stage closes only when all have decided, so a rejection carries every reviewer's findings. A new version during an open stage invalidates it (`ReviewsInvalidated`) and reopens the agent stage. A decision on a closed stage is refused. Human stages open only after agent stages pass.

Artifact	Agent stage	Human stage
Charter	—	Product Owner
Constitution	—	Solution Architect, Enterprise Architect
Feature Spec	format verifier only	Product Owner
Plan	analyzer; architecture, data, security, (UX) reviewers	Solution Architect
Amendment	—	Solution Architect, Enterprise Architect
Independent Data Model change	data, architecture, (security) reviewers	Solution Architect for BREAKING
Integration	full verifier run	Solution Architect for MEDIUM/HIGH

Workflows — Clarification

workflows/clarification.md

A role that cannot proceed without an answer it does not own opens a bounded session. See [../ledger/clarification.md](#) for the artifact.

asker's turn: runs the clarify skill → up to 5 questions with options and a recommendation

- POST /v1/clarifications { artifact, version, target, questions[] }

- ClarificationOpened; artifact reports waiting_on: <target>; turn ends

target (human, in the UI; or a role, as a turn):

- answers each question (or "let the asker decide" for a role target)

- ClarificationAnswered → platform enqueues <asker>.clarify-apply

asker's turn: writes answers into ## Clarifications and the sections they resolve, submits a new version, POST /v1/clarifications/{id}/apply { version_id }

- ClarificationApplied; waiting_on clears

Rule	Enforcement
$\leq$ MAX_CLARIFY_QUESTIONS per session	refused at open
$\leq$ MAX_CLARIFY_SESSIONS per artifact lineage; $\leq$ MAX_REVIEW_QUESTIONS per reviewer per Plan version	refused at open; the asker records assumptions instead
No submit-for-review with an answered, unapplied session	refused
No status change	waiting_on is derived
A session outlives nothing	abandonment cancels it

Escalation chain. A planner's question the analyst cannot answer from the approved spec causes the analyst to open its own session with the Product Owner; the answer flows back as a spec revision. If the revision changes a requirement, the spec re-enters the Product Owner's approval and the Plan waits; if it only clarifies, the spec version is re-pinned without a new approval and the planner's clarify-apply proceeds.

Workflows — Review and Rework

workflows/review-and-rework.md

How a Plan moves through the analyze gate and the review round, and what a rejection costs.

```
planner submits bundle v1
  deterministic gates – refuse → fix within the turn → resubmit
  [ANALYZING] – analyzer turn
    CRITICAL → [REJECTED] (analyze_returns++) → plan.revise → v2 → ...
    else → [UNDER_REVIEW] agent stage: review turns in parallel
      each: checklist + findings (+ patch) (+ one bounded session with the planner)
      all decided:
        any REJECT → PlanRejected(all findings, all patches) (rejection_count++) → plan.revise → v2
        all APPROVE → human stage: Solution Architect
          REJECT (reasons) → (rejection_count++) → plan.revise
          APPROVE → PlanApproved, TasksCreated → backlog
```

Revision. `plan.revise` starts from the previous version, the analyze report, every verdict, and every proposed patch. The skill applies patches it agrees with, argues in `research.md` where it does not, and resubmits the whole bundle. Reviews on the old version are invalidated; the analyzer runs again; the agent stage reopens. Reviewers see the diff between versions and their own previous verdict.

Bounds. `MAX_ANALYZE_RETURNS` stops the planner-analyzer loop and hands the Plan to the Solution Architect with the report. `MAX_PLAN_REJECTIONS` stops the review loop the same way, with three choices: redirect, Amendment, or abandon. A redirect is a recorded note the planner revises against and resets the count.

What reviewers may not do. Reject for brevity; ask for elaboration of a negative determination; ask more than `MAX_REVIEW_QUESTIONS`; leave a checklist item unanswered; reject without a HIGH or CRITICAL finding.

Workflows — Blocked

workflows/blocked.md

Blocked is a condition on an artifact, not a status. A blocker is a row in the registry naming the artifact, the reason, the raiser, and the human role that owns clearing it.

raiser (a turn or the platform) → BlockerRaised { artifact, reason, resume_context }
platform:
 machine-resolvable? (a predecessor now DONE, a reservation released, a stand-down cleared,
 a budget raised) → BlockerCleared(cleared_by: platform) immediately
 else → owner_role by rule → that human's inbox
human clears with a resolution → BlockerCleared → the platform enqueues the turn that resumes
 the work, carrying resume_context

Raised by	Typical reason	Owner
implementer turn	the Plan is ambiguous at a point a task needs	Delivery Lead (usually becomes a Defect)
verifier	MAX_VERIFY_ATTEMPTS, MAX_INTEGRATION_ATTEMPTS	Delivery Lead
platform	BUDGET_EXHAUSTED	Product Owner
platform	DeploymentFailed	Delivery Lead
platform	reservation collision at integration	Delivery Lead
Solution Architect	integration risk rejected	Delivery Lead

While blocked, an artifact keeps its status and its tasks stay off [READY]; the scheduler holds a blocked feature; the dashboard shows `blocked: true` with the reason and owner. Clearing resumes exactly where the raiser stopped, because the resuming turn is derived from the artifact's state and the recorded context.

Workflows — Abandonment

workflows/abandonment.md

Only a human retires work. The Product Owner abandons a Feature; the Delivery Lead or Product Owner abandons a task. A reason is required. [ABANDONED] is terminal.

Feature. FeatureAbandoned on the feature stream: the Spec, Plan, and every non-[DONE] task go to [ABANDONED]; every queued turn on the feature is cancelled and every claimed one has its token refused at its next write; open clarification sessions, blockers, and review stages close; Data Model reservations lapse; the feature's citations stop counting; the branch is kept for audit and excluded from the merge queue; the budget ledger closes. A feature in [INTEGRATING] that has already merged cannot be abandoned — it is [DONE], and its removal is a new Feature.

Task. TaskAbandoned: the task and its transitive successors go to [ABANDONED]; their turns are cancelled. The Plan's coverage is now incomplete, so the platform raises a Defect to the planner naming the abandoned tasks; the feature cannot integrate until the Plan is revised to drop or replace them.

Amendment. The architect's proposal may be abandoned by either approving human; a Plan waiting on it returns to the planner with the reason.

Workflows — Amendment

workflows/amendment.md

trigger: analyze gate finds a Constitution gap (CRITICAL: capability not granted / Article conflict)
 or the architect proposes independently
platform enqueues amend.author (architect); the triggering Plan waits at [DRAFT] with waiting_on: amendment
architect turn: proposes A-#### with the diff, the level, and the impact list (computed by the platform
 from Required Capabilities and Constitution Checks of every open Plan)
submit → [UNDER_REVIEW]: Solution Architect and Enterprise Architect decide
 both APPROVE → applied in the same transaction: new Constitution version, semver bumped,
 AmendmentApplied(affected_features[]), [APPLIED]
 → plan.analyze enqueued for every affected open Plan
 → the triggering Plan resumes: plan.revise enqueued
either REJECT → [REJECTED]; the triggering Plan is [REJECTED] with the reason;
 the planner revises within the Constitution, or the analyst revises the spec,
 or the Product Owner abandons

A Plan already in Build whose re-analysis returns CRITICAL is held at the integration gate for the Solution Architect, who accepts it against the old Constitution version (recorded) or sends it back as a Defect.

Workflows — Data Model Change

workflows/data-model-change.md

Feature-driven. The planner's `data-model.md` cites and declares. At Plan submit the platform validates declarations, reserves introduced names, classifies against the current model, and refuses BREAKING. The data reviewer reviews the design; the Solution Architect approves it with the Plan. At integration the platform re-classifies (the model may have moved) and transcribes: `DataModelTranscribed`, each document [APPROVED] with `feature_id` and `plan_version_id`. A BREAKING result at integration raises a Defect and refuses the merge; a name that another feature landed first raises a reservation blocker to the Delivery Lead.

Defect touching the data design. A Plan revision in Build that changes a declaration reopens the data and architecture reviews (and security when a sensitivity moved); the revised declaration is re-classified and re-reserved; tasks depending on it wait.

Independent. The reviewer.data role proposes a new document version directly (`DataModelChangeProposed`) with the platform-computed classification and impact list. Reviews: architecture always, security for sensitivity, Solution Architect for BREAKING. Approval transcribes immediately and, for BREAKING, raises a Defect on every live Plan in the impact list so each one revises its citations.

Deprecation and retirement. Deprecation is an independent change that lists live citing features; retirement is refused while any exist.

Skills

skills/README.md

A skill is what a worker loads to execute a turn of a given kind. Skills are versioned repository content; the platform records `skill@version` on every turn, so every artifact's audit trail names the instructions that produced it.

Skill contract

skills/<name>/	
SKILL.md	front matter + instructions
templates/	artifact templates the skill fills
checklists/	rubric answered item by item (reviewer skills)

Front matter:

```

name: plan
version: 1.4.0
role: planner
kinds: [plan.author, plan.revise]
model_tier: reasoning
needs_fs: true
inputs: [spec@approved, constitution@current, data-model, plan@previous?, analyze-report?, verdicts[]?]
outputs: [plan-bundle]
commits: submit-plan # the turn's single committing write
tools: [read-artifact, submit-plan-version, submit-plan, open-clarification, read-data-model]
expected_seconds: 600 # the platform sets the lease to 3× this
slots: 2 # worker capacity one turn of this kind occupies
checkpoints: [research, data-model, contracts, threat-model, quickstart, tasks]
output_schemas: # validated in-turn, per call, before anything is submitted
  plan.md: schemas/plan.schema.json
  data-model.md: schemas/declaration.schema.json
repair_passes: 2 # bounded; a non-converging repair fails the turn
budget: { tokens: 400000, cost_micros: 6000000 }
verifier: [format, capabilities, constitution-check, coverage, data-model, write-set, markers]

```

The platform validates on registration that every kinds entry maps to role, that tools are within the role's permission set, that commits names exactly one write, and that verifier names checks the runner has. A worker refuses a turn naming a version it does not have.

Four fields exist for reliability under load rather than for quality:

Field	What it buys
commits	One committing write per turn, so a turn cannot half-create an artifact
expected_seconds	A lease sized to the turn, so a long turn is not reaped and a dead short one is not held
checkpoints	A requeued attempt resumes instead of re-deriving, so a crash costs minutes rather than a whole turn's tokens
output_schemas, repair_passes	A malformed or truncated model output is repaired inside the turn, so it costs one small call instead of a requeue that will truncate again

Skills built on Spec Kit's command templates say so, so that upstream template changes can be reviewed against the skill.

Catalog

Skill	Role	Kinds	Tier	Produces	Built on
charter	analyst	charter.interview, charter.author	reasoning	charter.md through a guided interview with the Product Owner	AENEAS charter template
specify	analyst	spec.author	reasoning	spec.md	Spec Kit specify
clarify	analyst, planner, reviewers	spec.clarify, spec.clarify-apply, plan.clarify-apply	standard	a session; then the applied version	Spec Kit clarify
constitution	architect	constitution.author	reasoning	constitution.md from the approved Charter	Spec Kit constitution + AENEAS sections
amend	architect	amend.author	reasoning	A-####.md with diff, level, impact	—

Skill	Role	Kinds	Tier	Produces	Built on
plan	planner	plan.author, plan.revise	reasoning	the bundle; calls data-model, contracts, threat-model, quickstart, tasks, checklist as sub-skills	Spec Kit plan
data-model	planner (sub-skill)	—	reasoning	data-model.md with declarations	Spec Kit plan phase 1 + representation contract
contracts	planner (sub-skill)	—	standard	contracts/	Spec Kit plan phase 1
threat-model	planner (sub-skill)	—	reasoning	threat-model. md with contro lCoverage	AENEAS security model
quickstart	planner (sub-skill)	—	standard	quickstart.md with assertions	Spec Kit plan phase 1
tasks	planner (sub-skill)	—	reasoning	tasks.md	Spec Kit tasks

Skill	Role	Kinds	Tier	Produces	Built on
checklist	planner, reviewers	—	fast	checklists/*.md	Spec Kit checklist
analyze	analyzer	plan.analyze	fast	analyze report	Spec Kit analyze
review-architecture	reviewer.architecture	review.architecture	standard	verdict	checklist
review-data	reviewer.data	review.data, data-model.propose	standard	verdict; independent change	checklist
review-security	reviewer.security	review.security	standard	verdict	checklist
review-ux	reviewer.ux	review.ux	standard	verdict	checklist
test	tester	task.test	standard	acceptance and contract tests	Spec Kit implement (test phase)
triage	tester	task.triage	fast	Bug or Defect decision with evidence	—
implement	implementer	task.implement, task.setup, task.polish	reasoning	code and unit tests on the branch	Spec Kit implement

Skill	Role	Kinds	Tier	Produces	Built on
implement-frontend	implementer	task.implement (UI paths)	reasoning	UI code	Spec Kit implement
implement-data	implementer	task.foundation (migrations)	reasoning	migrations, data access	—
fix	implementer	task.fix	reasoning; escalates on attempt 3	a fix for the failing check	—
defect	planner	defect.resolve	reasoning	a revised bundle	plan
integrate	integrator	integrate	standard	a rebased, pushed branch	—
release	integrator	release, rollback	standard	deployment signals	project pipeline
observe	operator	observe	fast	anomaly records	—

Exemplars

- [plan/SKILL.md](#) — the planner's skill, in full
- [review-data/SKILL.md](#) — a reviewer skill with its checklist

SKILL

skills/plan/SKILL.md

name: plan version: 1.4.0 role: planner kinds: [plan.author, plan.revise] model_tier: reasoning needs_fs: true inputs:

- spec@approved
- constitution@current
- data-model@current
- plan@previous # plan.revise only
- analyze-report # plan.revise only
- verdicts[] # plan.revise only, with patches
- redirect-note # after MAX_PLAN_REJECTIONS, if the Solution Architect redirected

outputs: [plan-bundle] commits: submit-plan tools:

- read-artifact
- read-data-model
- read-repository # trunk, read-only: structure, existing contracts, existing tests
- submit-plan-version
- submit-plan
- open-clarification # target: analyst

sub_skills: [data-model, contracts, threat-model, quickstart, tasks, checklist] expected_seconds: 600 slots: 2 checkpoints: [research, data-model, contracts, threat-model, quickstart, tasks] output_schemas: plan.md: schemas/plan.schema.json data-model.md: schemas/declaration.schema.json tasks.md: schemas/tasks.schema.json repair_passes: 2 budget: { tokens:

400000, cost_micros: 6000000 } verifier: [format, capabilities, constitution-check, coverage, data-model, write-set, markers]
built_on: spec-kit/templates/commands/plan.md,
spec-kit/templates/plan-template.md

plan

You are the planner. You produce the complete Plan bundle for one feature in this turn. Nobody else authors any part of it; specialists will review it.

Read

1. specs/F-####-slug/spec.md at the approved version. Every user story, FR-####, SC-####, constraint, and assumption is an obligation.
2. constitution.md. Every Article is a gate you must pass or justify. The Capability Map is the only vocabulary ## Required Capabilities may use. The Data Design Language's representation contract is the schema every declaration must satisfy. The Security Model's catalog is what threat-model.md maps controls to.
3. The project Data Model: cite what exists, declare only what the feature needs.
4. The trunk (read-only): existing modules, contracts, test layout. Extend what exists before adding what does not.
5. On plan.revise: the previous bundle, the analyze report, and every verdict with its patch. Address every finding; apply patches you agree with; where you do not, record why in research.md.

Produce

Fill templates/plan-template.md, then run the sub-skills in this order: data-model, contracts, threat-model, quickstart, tasks, checklist. Each sub-skill reads what the earlier ones produced.

- **Technical Context:** every field concrete; a field you cannot fill is a NEEDS CLARIFICATION you must resolve in research.md (decision, rationale, alternatives) before submitting. The verifier refuses a marker left anywhere in the bundle.
- **Constitution Check:** every Article listed, PASS / JUSTIFIED / FAIL. A FAIL is not submittable. A JUSTIFIED needs a Complexity Tracking row. If the feature genuinely needs what the Constitution forbids or does not grant, stop: state the gap in plan.md under ## Constitution Gap and submit; the analyze gate will route it to an Amendment.
- **Required Capabilities:** identifiers copied verbatim from the map.
- **Write Set:** every path the tasks will touch, as globs, and nothing more. The scheduler holds features that overlap; the verifier fails diffs outside it. Wide sets delay scheduling; narrow sets fail verification. Be exact.
- **Proportionality:** a concern the feature does not engage is one line naming the trigger it does not meet. threat-model.md always names every surface in the Constitution with a determination.
- **Coverage:** every FR-### and story maps to tasks; every SC-### maps to a quickstart.md assertion or a checklist item. The verifier computes this; do not submit with a gap.

Ask

Open one clarification session with the analyst only when the approved spec is ambiguous in a way that changes a task, a contract, or a declaration. Design decisions in your own discipline are yours. State assumptions in `## Assumptions of plan.md`. Sessions are bounded; past the bound, decide and record.

Checkpoint, and stop when told

Checkpoint after each sub-skill, naming the stage. If this turn dies, the next attempt is handed what you stored and continues from there — an unstored `data-model.md` is one you will pay to write again.

Heartbeat while you work. If a heartbeat answers anything but `CURRENT`, stop **before your next model call** and fail the turn with that reason: the spec has been revised or the feature abandoned underneath you, and everything you write from here is discarded. Watch `budget_remaining` and shorten your own work if it is running low — drop an optional research alternative, not a required section — rather than being failed mid-call at the ceiling.

Submit

Your turn has exactly **one committing write**: `submit-plan`, carrying every file of the bundle. Build the whole bundle first, validate each file against its schema as you produce it (a malformed or truncated output gets one bounded repair pass, not a resubmission), and submit once. There is no partial state to leave behind and no second call to make.

If the deterministic gates refuse, fix and resubmit within this turn; a refusal is not a committing write. Your turn ends at a successful submit, an open clarification session, or a failure.

You may not

Change `spec.md`. Write to the trunk. Name a capability the map lacks. Assert a `change_kind`. Leave a task without a file path. List an implement task before the test tasks of its story. Make a second committing write. Continue after a heartbeat says you are superseded.

SKILL

skills/review-data/SKILL.md

name: review-data version: 1.2.0 role: reviewer.data kinds: [review.data, data-model.propose] model_tier: standard needs_fs: false inputs: [plan-bundle@version, constitution@current, data-model@current, analyze-report] outputs: [verdict] tools: [read-artifact, read-data-model, submit-verdict, open-clarification] budget: { tokens: 120000, cost_micros: 900000 } verifier: [verdict-format, checklist-complete] checklist: checklists/data.md

review-data

You review the data design of one Plan version. You did not write it, and you have no memory of any earlier version except what the platform hands you. Answer every checklist item; the platform refuses a verdict with one missing.

Decide

- **APPROVE** when every item passes or fails only at **LOW/MEDIUM** severity with a recommendation the planner can take or leave.
- **REJECT** when any item fails at **HIGH** or **CRITICAL**: a citation that does not resolve, a declaration violating the representation contract, an attribute holding personal or regulated data without a sensitivity, a migration that destroys data without saying so, a design that duplicates an existing entity under a new name.

- Attach a **proposed patch** for every finding you can fix mechanically. The planner applies patches in its revision; a patch saves a round.

Ask

If the Plan is silent where the checklist needs an answer, open one clarification session with the planner (bounded). If the Plan is wrong, reject. Never ask for elaboration of a stated negative determination; never reject for brevity.

checklists/data.md

- [] D1 Every cited entity exists in the Data Model at [APPROVED]
- [] D2 Every introduced or revised document carries a declaration satisfying the representation contract
- [] D3 No introduced name duplicates an existing concept under another name (ubiquitous language)
- [] D4 Every attribute holding personal, financial, or regulated data declares sensitivity
- [] D5 Identity, aggregate membership, and lifecycle are stated for every entity
- [] D6 Relationships state cardinality and the owning side
- [] D7 The design's change_kind as the platform computed it matches what the prose claims
- [] D8 Migrations in tasks.md are reversible, or research.md says why not and how data is protected
- [] D9 quickstart.md exercises every introduced entity at least once